# Part II

# ML-powered plugin control: the meta-controller

# 10

# *Using regression for synthesizer control*

This chapter introduces regression, a statistical technique to estimate values between known data points. The chapter starts with 2D linear regression, fitting a line to known data points. Then, it expands the concept to higher dimensions to enable one-to-many mappings. I explain how this relates to sound synthesiser control. I will show you how neural networks can implement the same line equation as linear regression, where weights and biases are the parameters for the network, and learning is the process of estimating suitable values. At the end of the chapter you should have an understanding of how linear regression and simple neural networks work.

## 10.1 What does the meta-controller do?

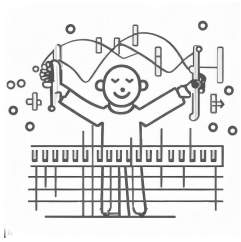

**FIGURE 10.1**
The meta-controller uses regression to control other plug-ins.

Now that you have completed installing, testing and familiarising yourself with the book's core tools, you will work towards the first serious example project: the meta-controller. The meta-controller is an innovative tool for synthesizer programming. It allows you to control multiple parameters of a synthesizer using one or more controls, for example a single slider can control tens or hundreds of parameters. This means synthesizer users can move through the sound space of any plugin synthesizer more dynamically than they can via conventional synthesizer programming techniques. The meta-controller uses interactive machine learning to allow users to create complex linear or non-linear mappings between arbitrarily dimensional controls and synthesis parameters.

But before you dive in with the meta-controller project, I would like to explain the background machine learning technique it uses. The technique is called regression, and we'll do it using a neural

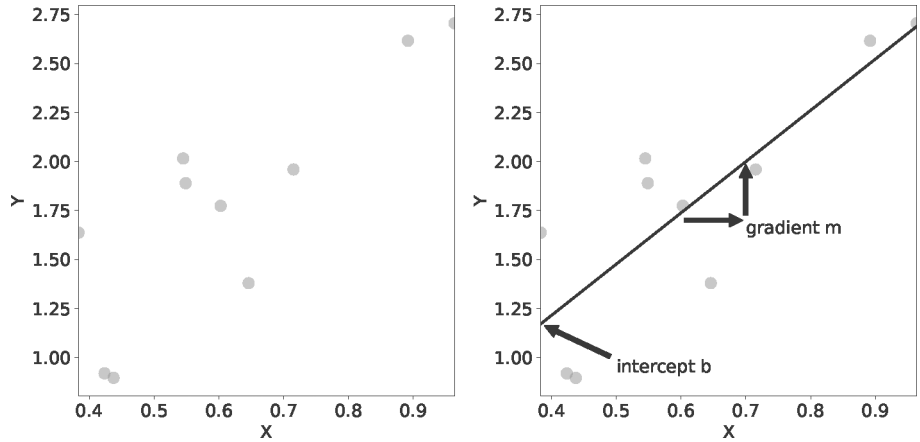


**FIGURE 10.2**
Linear regression finds the straight line that best fits some data. Important features of the line are the point at which it intercepts the y-axis and the slope gradient.

network. The aim for this chapter therefore is to explain what regression is and how to do it using neural networks.

## 10.2 Linear regression and synthesizer control

The meta-controller is an AI-enhanced synthesizer programmer which allows you to use simple controls to move through the sound space of any plugin synthesizer more dynamically than you can via conventional synthesizer programming techniques.

The meta-controller uses a technique called regression. Regression is a term from statistics that most people encounter when they learn about making predictions from graphs. The idea is that you have data for several points on a graph, but you want to know what happens between or beyond those points, so you can estimate the value at a given position for which you do not have an actual data point. Linear regression is perhaps the most straightforward method to estimate what happens between and beyond the known data points. Linear regression involves fitting a line to the available points in the best way possible.

The logical next question is – how do you work out where to put the line? First, you can consider that you can specify a line as follows:

$y = mx + b$

$y$ is the y coordinate, $x$ is the x coordinate, $m$ is the gradient of the slope and $c$ is the intercept on the y axis. These values are illustrated in Figure 10.2. So the main job of a linear regression function is to compute $m$ and $b$. If you have $m$ and $b$, you can calculate $y$ for any $x$ you like. The best line will have the smallest total error between itself and the set of known data points. If the points are not on a straight line, the best line will not go through all the points, so the linear regression function has to minimise the error between the points and the line. Once you have estimated the intercept and gradient values $a$ and $m$ that define the line, you can estimate any $y$ given an $x$ value. Simple!

### 10.2.1 Regression or interpolation?

Regression and interpolation are related processes that aim to determine what the data does between the known points. The difference is that interpolation requires that the resulting line goes through all the data points, and regression does not. Simple linear interpolation is not suitable for noisy data as a single line cannot pass through all the data points. There are more complex forms of interpolation that can handle more complex data, but for now, we will focus on the more 'heuristic' regression.

### 10.2.2 Regression and synthesizer control

What does this have to do with synthesizer parameter control, though? As mentioned above, the meta-controller lets you control several parameters from a single controller. If the meta-controller worked like the simple linear regression example above, it would estimate one parameter (the $y$ value) given the control value (the $x$ value). That is probably pointless – it is not much use to map one parameter to one other parameter. What we want to do with the meta-controller is map one parameter (e.g. a single slider control) to control many parameters on the synthesizer. To do that, we need to go into higher dimensions. That sounds mind-bending, but all it means is that we are estimating more than one line. So the slider value ($x$) maps to multiple lines or multiple parameters. Each line has its own intercept point and gradient, allowing differently calibrated control of each parameter. Figure 10.3 illustrates how, with two lines, we can estimate values for two parameters given a single input. This mapping can be referred to as one-to-many, as one value controls several others.

One-to-many mapping is already possible in commercial synthesizers. The 'Super-knob' on the Yamaha ModX synthesizer can be mapped to multiple parameters. You can map a single knob control (the Super-knob) such that it controls

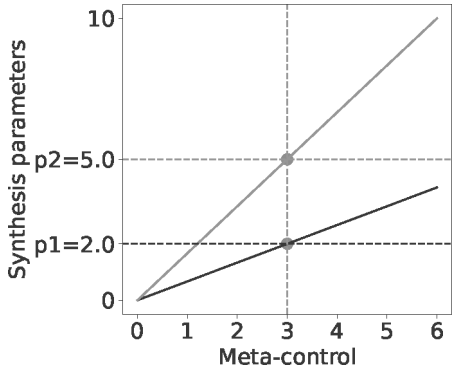


**FIGURE 10.3**
Linear regression with two lines, allowing the estimation of two parameters given a single 'meta-controller' input control. The x-axis represents the control input and the y-axis shows the settings for the two parameters the control is mapped to.

several other parameters simultaneously. Other synthesizers permit this kind of mapping with varying degrees of sophistication; for example, my Access Virus C synthesizer allows me to map two knobs on the control panel to various synthesis parameters via its mod matrix feature. But the Access Virus only allows me to specify the multiplier $m$ and not the intercept point (i.e., the control range over the other parameters). Have you used a synthesizer which provides this kind of meta-control? If you want to try it out, go back to the FM synthesis example plugin and add a new controller that changes the modulation index and modulation at the same time.

The meta-controller is a relative of the Super-knob, but it is more powerful and intuitive. More powerful because the meta-controller allows a user to create linear or non-linear mappings, unlike the more common linear mappings. And it can map between arbitrarily dimensional controls, meaning you can map an x,y pad to control 10 parameters, or a single slider to control 100 parameters, or any other combination. This is many-to-many mapping. The meta-controller is more intuitive because it uses a 'learn by example' workflow, sometimes referred to as *interactive machine learning*[34]. This means you can present it some examples of the mapping you want, and it will learn the configuration. I will talk more about interactive machine learning when we start work on the meta-controller; let's get back to the topic of linear regression with neural networks.

## 10.3 Neural networks and linear regression

Now that you understand what linear regression is (estimating the slope and intercept point of one or more lines) and how it can be used for synthesizer control (mapping between parameters), it is time to learn how to carry out linear regression using a neural network. Those familiar with statistical analysis using languages such as R and Python might think that a neural network is an overly sophisticated tool for this task. There are a couple of reasons for using neural networks though: 1) they provide a convenient workflow for users to iteratively specify a mapping, and 2) unlike more straightforward methods, they can scale to the more demanding task of many-to-many non-linear mapping that we might want to carry out later, with a very similar workflow. The scalability has been one of the drivers of success for neural networks.

## 10.4 Feed-forward neural networks

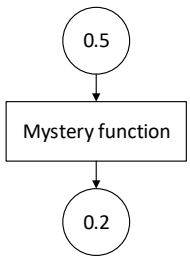


**FIGURE 10.4**
A neural network applies a function to its input.

A common type of neural network is a feed-forward neural network. Numbers go in one end, and numbers come out of the other in response. From the outside, it is just like a function in C++ where the inputs are the function parameters and the outputs are the return value. Unlike a regular function such as those you might write in C++, a neural network starts off not knowing what exact computations it needs to do to get from the input to the output. It must *learn* what the computation is. Learning functions is what machine learning is all about. But we should not get ahead of ourselves. Before we can consider how a neural network might learn, we need to think about what it is made of and how that makeup allows it to calculate the outputs based on the inputs. Consider the simple feed-forward neural network shown in figure 10.4.

What is the mysterious process? It turns out that in simple feed-forward neural networks, the process is just the same as the definition of a straight line, in other words, the output $y$ is the input $x$ scaled and summed with another value ($y = mx + b$). The difference is that we change the names of the variables. The scaling value $m$ is called a *weight* and the adding value is called a *bias*. So $y = wx + b$.

Given an input of 0.2, what should the weight $w$ and the bias $b$ be to produce an output of 0.5? Can you come up with a solution? You probably realise there are many solutions. I came up with $w = 1$ and $b = 0.3$. That works, but what if we have a second example of an input-output pair, such as 0.4 and 0.9? Do my values for $w$ and $b$ still work? Do yours?

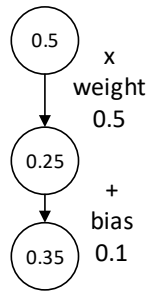


**FIGURE 10.5**
A neural network scales by a weight and adds a bias.

What kind of process did you follow to figure out the weight and bias parameters? In neural networks, the process is referred to as learning and it involves iteratively adjusting the weight and the bias until the network generates correct or close to correct outputs when presented with the inputs. When I told you about the second input-output pair, you may have changed your estimated weight and bias. If you did, you were acting like a neural network! Of course, real neural networks have many weights and biases, allowing them to model sophisticated functions. The weights and biases are referred to as parameters. Our network from figure 10.5 had two parameters. For some perspective, consider that we count the number of parameters for large language model neural networks such as the GPT series in the *billions* or *trillions*. But they still fundamentally take an input and process it in some way to produce an output.

## 10.5 Progress check

You should now understand what 1D linear regression is and how it can be expanded to higher dimensions. You should also understand the connection between simple feedfoward neural network units with weights and biases and linear regression.

# 11

# *Experiment with regression and libtorch*

Now you have some feel for the way a neural network might model a linear relationship between inputs and outputs, it is time to dive into an implementation of this using libtorch. This chapter provides instructions for implementing linear regression with a simple neural network using libtorch. It assumes that the development environment is set up, including an IDE, JUCE, CMake and libtorch. The implementation involves creating a new project, generating a dataset, adding noise to the dataset, creating a neural network model, converting the dataset to tensors, computing the error, learning from the error using the optimiser, and running the training loop. So you are going to be busy. The result of working through this will be a trained neural network that can predict y values from x values with a low error rate. The instructions also include challenges to extend the implementation to higher-dimensional datasets and non-linear mappings.

## 11.1 Create a new project

Start by creating a new project folder. You can re-use the CMakeLists file from your minimal libtorch project, which was based on repo-guide 39.2.10. You should have a folder structure as follows:

```
.
|-- CMakeLists.txt
`-- src
    `-- main.cpp
```

Your main.cpp file should be the same as the minimal libtorch example:

```
// I have added some extra includes that we will need later
#include <vector>
#include <iostream>
#include <random>
#include <torch/torch.h>

int main() {
    torch::Tensor tensor = torch::rand({2, 3});
```

```
    std::cout << tensor << std::endl;
}
```

Note the extra includes I added at the top. We will need those shortly. I will not repeat the CMakeLists.txt file here for brevity, but make sure you have the extra lines for libtorch as seen in listing 39.2.10 in the repo guide.

## 11.2 Create a dataset

First, you need to create a dataset containing x,y coordinates. You will use this to train and test the neural network. The dataset should be suitable for line fitting via linear regression. Therefore you need to generate the points along a line with some random noise added. Some points should fall above and some below the line. The dataset-generating function should allow the line to be offset (or biased) such that it crosses the y-axis at a certain point and allow the y values to be a consistent multiple of the x values. Let's break that into steps. Step 1 is to generate a linear ramp of values. Here is a function signature for that:

```
/**
* @brief Generate x,y values on a linear ramp. y = m(x) + b
*
* @param b: bias
* @param m: multiplier
* @param startX: start of the value range
* @param end: end of the value range
* @param count : how many x,y pairs to generate
*/
std::vector<std::pair<float,float>>  getLine(float b, float m,
                            float startX, float endX, float count)
```

$std :: vector$ is a growable array. $std :: pair$ is a pair of values. $std :: vector < std :: pair < float, float >>$ specifies a growable array containing pairs of floats. In other words, our x,y values.

Now the implementation:

```
float y{0}, dX{0};
std::vector<std::pair<float,float>> xys;
dX = (endX - startX) / count; // change in x
for (float x = startX; x < endX; x+=dX){
    y = (m * x) + b;
    xys.push_back({x,y});
}
return xys;
```

Read it carefully. Can you write some tests to verify that it generates the numbers as expected? Here is an example of a test wherein I ask for 10 values with

x in the range of 0 to 10. Notice how I access the first x value with $xys[0].first$. To access the y value, you can use $xys[0].second$. This is because each item in the array is a std::pair, and we access the values in the pair with '.first' and '.second'. So to the test – if the first value is not 0, print an error:

```
std::vector<std::pair<float,float>> xys;
xys = getLine(5.0, 0.5, 0.0, 10.0, 10.0);

if (xys[0].first != 0.0) {
    std::cout << "start value was not zero as expected "<< std::endl;
}
```

I like to use informal unit testing when developing number processing programs. You will see more of that as we work through it.

### 11.2.1 Dataset experiments

There are some extensions you can try out here. Can you test if the function works correctly if you try to create a ramp that goes down instead of up? In other words, set m to a negative value. Can you write a function that prints the x and y values from the function? Here is a signature:

```
void printXYs(std::vector<std::pair<float,float>>& xys)
```

## 11.3 Add noise to the dataset

Now that we have a linear ramp of values, we need to make it more challenging for the neural network by adding noise. Here is a function signature:

```
void addNoiseToYValues(std::vector<std::pair<float,float>>& xys,
                       float low, float high)
```

For less experienced C++ programmers, note that we are sending in the 'xys' parameter, which is a vector of pairs. Also, I added the ampersand character & after the type to use 'pass by reference'. This means that C++ will pass in a reference to the xy values instead of making a copy and passing that. Using the ampersand avoids the costly copy operation, but we must remember that the function now has direct access to the original version of the data sent to it.

Here is an implementation using the random functions available in the standard library:

```
std::default_random_engine generator;
std::uniform_real_distribution<float> distribution(low, high);
auto rand = std::bind ( distribution, generator );
```

```
for (int i=0;i<xys.size();++i){
    float r = rand();
    std::cout << "rand: " << r << std::endl;
    xys[i].second += r;
}
```

Creating a new random number generator every time the function gets called is probably inefficient, but it keeps things simple for now. Try your function out in combination with the printXYs function that you hopefully wrote earlier:

```
int main()
{
    std::vector<std::pair<float,float>> xys;
    xys = getLine(5.0, 0.5, 0.0, 10.0, 5);
    printXYs(xys);
    addNoiseToYValues(xys, -0.1, 0.1);
    printXYs(xys);
}
```

I see output like this with my version of the printXYs function:

```
x: 0, y: 5
x: 2, y: 6
x: 4, y: 7
x: 6, y: 8
x: 8, y: 9
x: 0, y: 4.9
x: 2, y: 5.92631
x: 4, y: 7.05112
x: 6, y: 7.99173
x: 8, y: 9.00655
```

## 11.4 Progress check

At this point, you should be able to generate linear ramps of values in specified ranges with a specified number of x,y pairs. You should be able to add noise to the x,y pairs and print them out.

## 11.5 Finding the best-fit line using a neural network

Now you have a dataset generator, it is time to use libtorch to fit the line. There are a few parts to this:

1. The neural network model that will learn to predict y values from x values.
2. Code to convert our x,y data into tensors so the network can understand it.
3. The error metric that will compute the error over the dataset.
4. The optimiser that will use the computed error to update the neural network weight and bias.
5. The training loop that will iteratively compute and learn from errors.

## 11.6 Neural network model

First, the neural network model. Neural networks are organised into layers which contain nodes or 'neurons'. We only need a single node for 1D linear regression:

```
auto net = torch::nn::Linear(1, 1);
```

Yes – that is all. Torch's 'Linear' is a simple 'feed-forward' neural network layer. We have sent the Linear function the arguments 1 and 1, which means the layer has one input and one output. Figure 11.1 shows a 1-in, 1-out Linear layer.

According to the documentation for Linear[1], it "Applies a linear transformation to the incoming data: $y = xA^T + b$". Aside from the $T$ component, that expression should be pretty familiar – it is the same as the definition of a straight line, $y = mx + b$. A quick note about the libtorch documentation: the best strategy is to look things up in the Python PyTorch documentation as it has more detail and explanation than the libtorch C++ documentation. Once you have found and read about the functions you want in the PyTorch documentation, you can locate them in the libtorch documentation, as sometimes the names change for C++.

So you have created your first neural network — but what are its parameters? You will be using machine learning to learn these parameters.

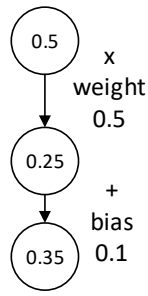


**FIGURE 11.1** Simple single layer network with one input and one output.

[1] https://pytorch.org/docs/stable/generated/torch.nn.Linear.html

You can print the parameters out as follows:

```
int main() {
    auto net = torch::nn::Linear(1, 3) ;
    for (const auto& p : net->parameters()) {
        std::cout << p << std::endl;
    }
}
```

On my system, this code outputs the following:

```
parameter:  0.3090
[ CPUFloatType{1,1} ]
parameter: -0.2448
[ CPUFloatType{1} ]
```

If you are working on Windows, you must ensure you are building in the same mode as the libtorch library you are using. If you are using the Debug version of libtorch, build in Debug mode, and the same for Release mode.

So we have two parameters – 0.3090 and -0.2448. Earlier I said that a single 'node' in a feed-forward neural network scales the input by a weight $w$ then adds a bias $b$. Let's pass some input to the network to verify that is indeed what it is doing. The following code passes a single value through the neural network by calling it as if it were a function. Add the code after your parameter printing code:

```
1 auto input = torch::empty({1, 1});
2 input[0][0] = 0.5;
3 auto output = net(input);
4 std::cout << "Passing 0.5 in ... this came out:\n"
5           << output << std::endl;
```

You might be wondering what empty is and what the data type of 'input' is. You cannot pass simple floats directly into a libtorch neural network – you have to wrap the floats inside a data structure called a tensor. Empty allows you to create an empty tensor of a certain shape (1x1 in this case). Then you can put data into the tensor using array-like syntax – check line 2. More on tensors in a bit but let's just see what comes out of the other end of the network. On my machine I see output like this, noting that the parameters are different every time I run it:

```
0.8999
[ CPUFloatType{1,1} ]
0.3680
[ CPUFloatType{1} ]
Passing 0.5 in ...
0.8179
[ CPUFloatType{1,1} ]
```

Let's verify we know what is going on. The input $x$ was 0.5; the output $y$ was 0.8179; the weight parameter $w$ was 0.8999 and the bias parameter $b$ was 0.3680. Use a calculator to confirm that $y = wx + b$.

### 11.6.1 More complex model

Just out of interest, let's see what a more complex neural network's parameters look like. Try changing the arguments you sent to Linear to be 2 and 3, to create a network layer like the one shown in figure 11.2. You will also need to change the code that creates the input. Here is the updated code to generate the input – note that the input tensor is now $1 \times 2$ in size:

```
auto input = torch::empty({1, 2});
input[0][0] = 0.5;
input[0][1] = 0.25;
```

On my machine I see the following output for the program:

```
0.0610  0.5951
-0.2292 -0.1606
-0.2962  0.5072
[ CPUFloatType{3,2} ]
-0.1701
    0.4497
    0.5419
[ CPUFloatType{3} ]
Passing 0.5 in ... this came out:
    0.0092  0.2950  0.5206
[ CPUFloatType{1,3} ]
```

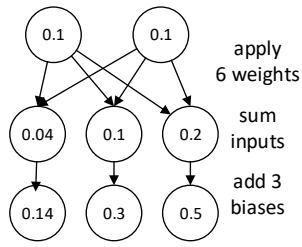


**FIGURE 11.2**
More complex single layer with more inputs and outputs. Now we apply a weight to each input as it goes to each output. We then sum the weighted inputs and apply a bias to each output.

The first iteration of the parameter printing loop prints six numbers. This is because each of the two inputs is connected to each of the three outputs, requiring six weights. Six connections, six weights. Each of those three outputs will sum its weighted inputs. The second iteration prints a further three numbers. Those are the biases applied to the outputs.

## 11.7 Converting the dataset to tensors

As you saw just now, to feed some data into the network, you must convert the data from float values into tensors. But what is a tensor anyway? Tensors are the life blood of neural networks, flowing through the network nourishing it with the data they carry. Perhaps that was a bit too colourful. The PyTorch documentation provides a succinct technical description of a Tensor[2]: "A torch.Tensor is a multi-

[2]https://pytorch.org/docs/stable/tensors.html

dimensional matrix containing elements of a single data type.”. If you were to put data types in order of increasing complexity, you would start with a single float, then a vector of floats, then a matrix and finally, a tensor.

The following lines create a tensor suitable for storing your vector of x,y pairs:

```
torch::Tensor in_t;
in_t = torch::empty({(long) xys.size(), 1});
```

Note that I specified the data type this time, instead of using auto. That makes it clear we are working with a Tensor variable. Then copy the x coordinates into the input tensor:

```
for (int i=0; i<xys.size(); ++i){
    in_t[i][0] = xys[i].first; // first is x
}
```

Now get ready to be impressed by how flexible neural networks are – we can pass the entire input dataset of x values into the network in one go, and it will compute the entire output dataset of its estimated y values:

```
torch::Tensor out_t = net(in_t);
std::cout << "output: " << out_t << std::endl;
```

We call our ‘net’ variable as if it is a function, passing it the input tensor. ‘net’ returns another tensor containing the network’s guesses as to what the outputs should be. Here is an example of the output I see from the above code:

```
output: -0.7382
0.6678
2.0737
3.4796
4.8856
[ CPUFloatType{5,1} ]
```

Notice that the shape (5 by 1) and type (CPUFloat) of the tensor and the data it contains are printed.

## 11.8 Computing the error

Now that we have the network output, we can compute the error. We are going to use mean squared error (MSE). Remember that the error is the difference between the output of the network and the output we want. The output we want is the correct set of y values. The error function needs the net output and the correct output as arguments, so we must convert the correct outputs into another tensor.

```
torch::Tensor correct_t;
correct_t = torch::empty({(long) xys.size(), 1});
```

```
for (int i=0; i<xys.size(); ++i){
    correct_t[i][0] = xys[i].second; // second is y
}
```

Then compute the error:

```
torch::Tensor out_t = net(in_t);
torch::Tensor loss_t = torch::mse_loss(out_t, correct_t);
float loss = loss_t.item<float>();
```

torch::mse_loss is how we calculate the loss. Try calculating the loss and then looking at the difference between the correct outputs and the outputs from the network. Can you figure out how MSE is being computed? Clue: MSE stands for mean squared error.

## 11.9 Learning from the error: the optimiser

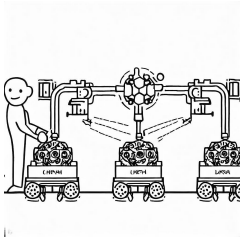

**FIGURE 11.3**
The optimiser adjusts the network weights.

Learning involves computing the error between the output of the network and the correct output, then adjusting the network's parameters so it can achieve a lower error. To do that, we need something called an optimiser. Various optimisers are available which deal with the learning process in different ways. For example, should the error across the whole dataset be considered or is it good enough just to look at a section of the dataset? How much should the network be adjusted on each iteration? Should the amount it is adjusted change over time, as the network zones in on a good set of parameters? Developing better optimisers, such as the Adam optimiser, was an important element in the rise of deep networks during the 2000s and 2010s.

We will use the stochastic gradient descent (SGD) optimiser. SGD is an optimiser that computes the changes needed to the network from a subset of the input-output pairs in the dataset. So it is quite an efficient optimiser. The following code creates an optimiser:

```
torch::optim::SGD optimizer(net->parameters(), lr=0.01);
```

Note how we pass the network parameters to the optimiser when we create it. To use the optimiser to train the network, we can use the following code:

```
// compute output
out_t = net(in_t);
// compute loss
torch::Tensor loss_t = torch::mse_loss(out_t, correct_t);
// reset the previous network parameter changes
optimizer.zero_grad();
// compute changes required to network parameters
loss_t.backward();
// update network parameters
optimizer.step();
```

This code represents training for a single epoch. Training for one epoch means you have calculated the error over the entire dataset. With larger datasets, the dataset is split into batches and the network is trained on one batch at a time. This makes it possible to train on larger datasets than you can store in memory. The linear regression dataset is very small so you can process the entire thing in a single batch.

## 11.10 The training loop

We must run the training process many times to properly train the network, i.e. achieve a low error. A low error means we have found a decent line fit for our dataset's x and y coordinates.

```
float loss = 1000;
while(loss > 0.5){
    torch::Tensor loss_t = torch::mse_loss(net(in_t), correct_t);
    loss = loss_t.item<float>();
    std::cout << "Loss: " << loss << std::endl;
    optimizer.zero_grad();
    loss_t.backward();
    optimizer.step();
}
```

Put this all together, organising it into functions if you wish to. Ensure you pass things like the network and the optimiser to functions 'by reference' (using &) to avoid expensive copy operations. You can find a solution to this in repo-guide 39.3.1. How many epochs does your network take to learn decent values for $w$ and $b$? How does adding more noise to the data affect the time taken to complete training? How low an error do you think you need? Note that it is not possible to achieve a zero error, i.e. a perfect line that goes through all the data points.

## 11.11 Training extension: higher dimensions

Can you generate a higher dimensional dataset and an appropriate network to carry out the regression? The example above maps from one-one — can you map two-one or many-many? You might find using the torch functions to generate higher-dimensional datasets easier than coding the dataset generators yourself. For example, check out torch's range[3] and rand[4] functions.

## 11.12 Progress check

At this point, you should be able to conduct a one-one linear regression using a torch Linear network layer. You should be familiar with datasets, training, errors and epochs.

[3] https://pytorch.org/docs/stable/generated/torch.range.html
[4] https://pytorch.org/docs/stable/generated/torch.rand.html

# 12

# *The meta-controller*

This chapter introduces the AI-music system called the meta-controller which you will be building for the next few chapters. The meta-controller system allows you to control many parameters on one or more instruments and effects simultaneously using a set of powerful 'meta-controls'. The meta-controller is a variant of Rebecca Fiebrink's Wekinator, built using C++ and the libtorch machine learning library; the difference is that the meta-controller is specialised for controlling standard plugin instruments and effects. This chapter describes the original Wekinator system and its workflow, as well as describing some previous variants that people have created such as the learner.js library. At the end of the chapter you should have a clearer idea about Wekinator and the meta-controller's interactive machine-learning workflows.

## 12.1 What are we going to build?

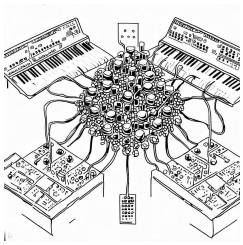

**FIGURE 12.1**
The meta-controller uses a neural network for new methods of synthesizer sound exploration.

The system we will build is a meta-controller for synthesizers and effects. In a sentence, a meta-controller is a system that allows us to control many parameters on one or more instruments and effects simultaneously using a smaller number of controls. For example, with a meta-controller, you can map positions on a laptop trackpad to presets on a synthesizer, then morph between these presets by moving between positions on the trackpad. The meta-controller can figure out what the synthesizer's parameters should be in between the presets you have mapped to the trackpad. In fact it can infer the synthesis parameters for any arbitrary position on the trackpad.

The meta-controller we create will be able to host native audio plugins. This means anyone using it can control whichever plugins they have installed on their system. The meta-controller will allow you to control more than one plu-

gin at a time, providing new modes of interaction with your plugins and breaking free from the limitations of graphical user interfaces and standard controllers.

## 12.2 Wekinator: inspiration for the meta-controller

The main inspiration for the meta-controller is Rebecca Fiebrink's 'Wekinator'. According to the Wekinator website[1], 'The Wekinator allows users to build new interactive systems by demonstrating human actions and computer responses, instead of writing programming code'. The concept is that you show Wekinator an input and tell it which output you want when it sees that input again. The clever part is that you can show it several input-output pairs, and after training, it can *infer* what the output should be when you show it an input it has not seen before. Returning to the example of mapping trackpad positions to synthesizer presets, you might place four different presets at the corners of the trackpad. Wekinator will generate and morph between the presets as you move around on the trackpad.

Fiebrink implemented Wekinator as an OpenSoundControl (OSC) message processor. The inputs are OSC messages, and the outputs are more OSC messages. The Wekinator system is flexible and comes with front-ends to convert various types of inputs (sensors, cameras, MIDI control messages, etc.) into OSC messages. It can then send its outputs as controls for any system that receives OSC messages (Ableton Live, SuperCollider etc.). Wekinator is built in Java and uses the Weka machine learning library[18], hence the name.

Since Fiebrink released the original version of Wekinator in 2009, there has been a Wekinator 2.0 and other versions. Louis McCallum worked with Fiebrink to reimplement the Wekinator concept in Javascript using the Tensorflow.js library. McCallum called the new library 'learner.js'[26]. Learner.js works in the web browser and is especially useful for folks wishing to share their creations with others. In fact learner.js was developed for use in the MIMIC[2] browser-based programming environment. MIMIC in combination with learner.js makes sharing and hacking other people's Wekinator examples much easier.

I worked with this team on the MIMIC project and have even performed live with Louis and MIMIC project lead Mick Grierson by collaboratively editing the same Javascript document! As a small anecdote, we agreed to do a live audiovisual performance using the MIMIC browser-based live coding environment during the 2020 Covid-19 lockdown. The performance was part of the concert programme for the NIME conference, which is perhaps the most important academic conference

[1] http://www.wekinator.org/
[2] https://mimicproject.com/

for folks developing innovative musical instruments. A few days before our performance, we discovered that our stream would be fed into a real Shanghai club containing real people we could not even see! Luckily nothing crashed too badly despite having three people madly editing the same file in real-time.

Returning to the central theme, we will build a version of Wekinator using C++ and the libtorch machine learning library. Our version will not be as general as Wekinator, and instead, it will be specialised for control of standard plugin instruments and effects. So it will be a plugin host that can learn how to control the plugins in response to different input types.

## 12.3 Inputs and outputs

Fiebrink describes Wekinator's inputs as 'human actions', and they can be complex things like a facial expression captured via a webcam or simpler things such as an x,y position on a laptop trackpad or a set of slider positions on an on-screen user interface. Whatever the human input, it ends up as an array of numbers inside Wekinator – a frame from a webcam is converted to a list of many RGB or greyscale values, and an x,y position on a trackpad becomes a list of two values.

The user can configure how many numbers Wekinator generates at its output based on their needs. For example, if they have a synthesizer with ten parameters and they want to control it using their trackpad, the input would have two numbers representing x and y on the trackpad, and the output would have ten digits, one for each synthesizer parameter.

## 12.4 The train-infer-train workflow

The workflow, illustrated in figure 12.2, is a significant innovation in the Wekinator system. It is described as interactive machine learning because the user presents training data to the machine learning algorithm in an interactive manner. Interactive means the user presents training data, trains in real-time, experiments with the trained model and then presents more data if needed. This is different from regular machine learning. In regular machine learning, the training is not interactive, and it generally involves a much larger dataset. Part of the trick here is knowing that you do not need perfect training with a huge dataset to provide a user with a sense of meaningful control. So long as something reasonably control-

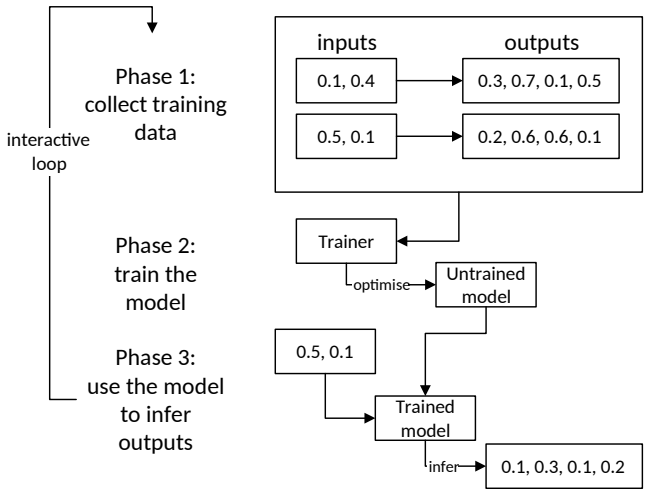


**FIGURE 12.2**
The Wekinator workflow: data collection, training, inference then back to data collection.

lable happens when the user manipulates the controller inputs, creative work can happen.

## 12.5 Wekinator's flexibility

Wekinator is a powerful system because of the interactive machine learning workflow and because it is a very general-purpose system. You can feed many types of input data to it – anything you can convert into OSC at any dimensional level. Then you can map the inputs to all kinds and dimensionalities of outputs. One problem with Wekinator is that its workflow is a little tricky to integrate with existing music technology. For example, to control a plugin synthesizer with Wekinator, you would need to figure out how to get your input control data into OSC messages and then convert the output OSC messages into control signals that the DAW hosting the plugin could understand. You would also need to specify how many parameters you wanted to control.

The meta-controller you will develop in the following sections sacrifices a little

of the flexibility of Wekinator to gain a deeper integration with standard music technology such as plugins and plugin hosts. But you could adapt it back towards the full Wekinator by adding OSC input and output to it.

## 12.6 Progress check

You should now be able to explain what the Wekinator is and how it enables its users to use an interactive machine-learning workflow to specify complex mappings between inputs and outputs. You should know that the meta-controller is a variant of the Wekinator which focuses on controlling plugins such as synthesizers and effects.

# 13

# *Linear interpolating superknob*

In this chapter, you will get started on the meta-controller system. The first step is implementing something similar to the 'superknob' found on Yamaha synthesizers such as the MODX+. The superknob described here allows you to control two synthesizer parameters on a simple FM synthesizer using a single knob. It does so using simple interpolation. This will lay the groundwork for the meta-controller, which allows you to control all parameters on any plugin using a single user interface control.

## 13.1 FM synthesizer

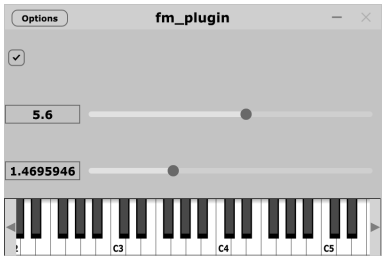


**FIGURE 13.1**
User interface for the simple two-parameter FM synthesizer. The toggle switch switches between drone and envelope mode, the two sliders control modulation depth and index and the piano keyboard allows you to play notes on the synthesizer.

You should start with the two-parameter FM synthesizer shown in figure 13.1, which can be found in repo-guide 39.2.9. To recap the features of this synthesizer:

1. Two sliders control modulation index and modulation depth
2. A toggle switch enables either constant playing or enveloped playing
3. A piano keyboard allows playing notes on the synthesizer

Make a copy of the project and edit the CMakeLists.txt file so it has your own 'company name' and plugin name/ code. Build and run it in standalone mode to verify it works correctly.

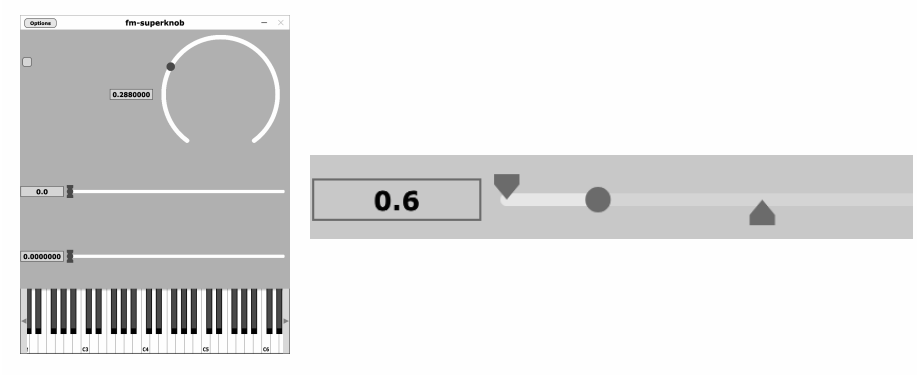


**FIGURE 13.2**
Superknob UI on the left. On the right is a closer view of a range slider. Small triangles above and below the line allow the user to constrain the range of the main slider control.

## 13.2 Implement the superknob

Now you have the FM synthesizer up and running, you are ready to implement the basic superknob. First, in the PluginEditor.cpp file, call the setSliderStyle function on the two-parameter sliders. Send it the argument juce::Slider::ThreeValueHorizontal. That will change the sliders from regular, single-position sliders into range sliders allowing you to constrain the value range. Figure 13.2 shows a range slider.

Next, you need to add the superknob to the user interface. This widget is just another slider, but you can call the setSliderStyle function to change it into a rotary 'knob' style. Use the juce::Slider::RotaryHorizontalDrag style. This knob can be adjusted by clicking and holding, then moving left or right, hence 'horizontal drag'. Remember the three-step process to add a widget to your user interface:

1. Add a juce::Slider member to the private section of the class in PluginEditor.h
2. In the constructor in PluginEditor.cpp, call addAndMakeVisible on the new widget and call addListener on the widget
3. In the resized function in PluginEditor.cpp, call setBounds on the widget to place it where you want on the user interface.

I made the superknob quite large for my user interface and placed it at the top of the interface, as shown in figure 13.2. You can design it however you like.

Now you have the superknob widget displayed on the user interface, you need to implement the listener code to respond to it and to cause a change to the two parameters. In the PluginEditor.cpp code, locate the sliderValueChanged function. Add an if statement to check if the incoming slider value is equal to the address of your superknob widget. In my case, the widget's variable is called superKnob, so I have some code like this:

```
if (slider == &superKnob){
//The user adjusted the superknob
}
```

Now your job is to take the value from the superknob widget in the range 0-1 and map it to the range specified by each of the sliders. This is just like the equation for a line: $y = mx + b$, where $y$ is the value for the slider (i.e. modulation index or modulation depth), and $x$ is the value of the superknob widget. To calculate $m$ (also known as the weight ), you need to work out the range for the target slider by subtracting its lowest allowed value from its highest permitted value. To calculate $b$ (also known as the bias), just use the getMinValue function:

```
double high = modIndexSlider.getMaxValue();
double low = modIndexSlider.getMinValue();
double weight = high - low; // m
double bias = low; // b
```

getMaxValue tells you where the user placed the high constraint on the three-point slider. getMinValue tells you where the user placed the low constraint. Now, you can plug weight and bias into the equation and send the value to the target slider:

```
double superV = superKnob.getValue(); // x
double newV = (superV * weight) + bias; // y
modIndexSlider.setValue(newV);
```

Repeat the code for the other slider, called modDepthSlider, in the FM synthesizer example code. You should now be able to move the superknob dial and see the two sliders moving in their constrained ranges. You now have your own superknob.

## 13.3 Progress check

You should be able to explain the concept behind the superknob and how it allows a single knob to control multiple parameters at the same time using range

controls amd linear interpolation. You should have an FM synthesizer plugin with a working superknob feature that controls two parameters at the same time in ranges that the user can specify.

# 14

# *Untrained torchknob*

This chapter provides instructions on replacing the linear interpolation system in the superknob control with a neural network. We shall call this new system the 'torchknob'. The chapter will guide you through creating a simple neural network that can map from a single input control to two output controls, then adding a softmax layer to manage the output ranges. You will create a separate NeuralNetwork class which is designed to provide functions for passing data and training the network. At the end of the chapter, you will be able to control the FM synthesizer controls using a neural network, though not in a very predictable way yet!

## 14.1 Torchknob software architecture

Figure 14.1 shows a high-level architecture for the torchknob system. This is structurally similar to the superknob from the previous section, but instead of using range controls to specify the mapping from the large dial at the top to the synthesizer parameter sliders at the bottom, the user trains a neural network to learn the mapping. The aim is to implement the Wekinator workflow mentioned previously:

1. Set the parameter sliders to the desired position
2. Set the torchknob control to the desired position
3. Store the torchknob → sliders mapping as a training data point
4. Back to step 1. until the user has defined as many input-output associations as they like
5. Train the neural network on the set of torchknob → sliders mappings
6. Move the torchknob at the top, and the neural network will automatically set the sliders to the desired positions, inferring or regressing between the points in the training data

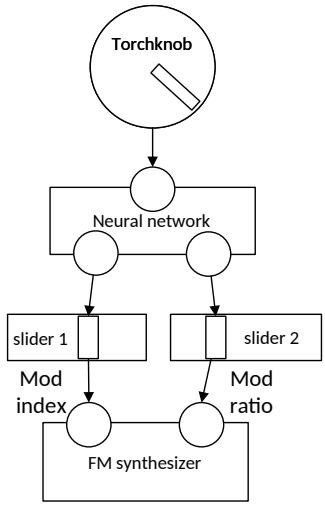


**FIGURE 14.1**
The torchknob system architecture.

## 14.2 Quick hack to pass data through a network

Start by making a copy of your FM superknob project. Remember to rename the plugin and give it a unique code so the new project does not clash with the other one in your plugin lists. Once you have a copy of your SuperKnob project, edit your CMakeLists.txt file to include libtorch in the project. You can find instructions on how to do this in section 5.9.

Next you can try a quick hack to pass data from one slider via a neural network into the other two sliders. In PluginEditor.h, make sure you include the torch/torch.h header. Then add this to the private section of the editor class:

```
torch::nn::Linear linear{1, 2};
```

That creates a class member with the type torch::nn::Linear and the shape: '1 input, two outputs'. Just what we need to map from one slider to two other sliders. Then you can use this code to pass the superknob value through the linear layer in the sliderChanged function in PluginEditor.cpp:

```
// create a 1x1 tensor
torch::Tensor in = torch::empty({1, 1});
//Copy the superknob value into the tensor
in[0][0] = superKnob.getValue();
//Pass the tensor through the linear layer
torch::Tensor out = linear(in);
// print the result (DBG does not work)
std::cout << out << std::endl;
//Extract the result from our tensor
// with out[0][1].item<float>()
// which specifies the position and
// data type you want to convert the tensor
// data into
modDepthSlider.setValue(out[0][0].item<float>());
modIndexSlider.setValue(out[0][1].item<float>());
```

Try moving the superknob around. You should see the two synthesis parameter sliders moving. I say 'should' because the network starts off with random weights and biases, so the two sliders might not move much or at all because the values coming out of the network are out of range of the slider. Each time you run the program, the network weights and biases (parameters) will be different. Keep re-running until you get some movement.

## 14.3 Basic NeuralNetwork class

Now you have a proof of concept in place; you can read values from a slider, feed them through a neural network, read the outputs and use them to set values on two other sliders. The problem is that the code will become messy once you progress onto more complex networks and once you want to start training. You will have a mix of user interface and neural network code in your PluginEditor files. The solution to this problem is to begin work on a separate NeuralNetwork class that provides a simple set of public functions for passing data in and training. When designing classes, I usually start with an 'aspirational' header file containing everything I wish I had (aside from a Lamborghini, of course). For example, in the header file, I specify the functions I would like to have. So let's do that. Create two new files in your src folder: NeuralNetwork.h and NeuralNetwork.cpp. In NeuralNetwork.h, put the following 'aspirational' class definition:

```
#pragma once

#include <torch/torch.h>

class NeuralNetwork : torch::nn::Module {
    public:
        NeuralNetwork(int64_t n_inputs, int64_t n_outputs);
        std::vector<float> forward(const std::vector<float>& inputs);
        void addTrainingData(
             std::vector<float>inputs,
             std::vector<float>outputs);
        void runTraining(int epochs);
    private:
        int64_t n_inputs;
        int64_t n_outputs;
        torch::nn::Linear linear{nullptr};
        torch::Tensor forward(const torch::Tensor& input);

};
```

Some comments on that code:

1. The NeuralNetwork class inherits from torch::nn::Module. This means it inherits all the functions of that parent class. Inheriting from Module is the recommended way to create a network in Torch.
2. The NeuralNetwork constructor takes two integers as arguments, specifying the number of inputs and the number of outputs.
3. The public forward function takes a vector of floats as input and provides a vector of floats at its output. Whilst the network itself will need the data in the form of tensors, from the JUCE side, it is cleaner if we work in floats and vectors as the JUCE code then does not need to know anything about libtorch.
4. There is a private function called forward, which receives and returns tensors. The other forward function will use this function internally to complete the inference.
5. Both forward functions specify that their arguments are const references (&). For those who are a bit new to C++, this means we avoid copying the incoming data every time the functions are called (pass by reference), and we are not allowed to change that incoming data (const).
6. There is an addTrainingData function which will enable us to add training examples (input-output pairs).
7. There is a runTraining function which will eventually allow us to train the network against the training examples.
8. There is a private class member called linear, a Linear network layer. It starts being set to a nullptr value.

You should now write comments above each function to explain what it does. Having written your documentation, it is time to turn your attention to the NeuralNetwork.cpp file. This is where we make good on the promises in the header file. You need to add NeuralNetwork.cpp to the CMakeLists.txt file in the target_sources command so that the new class is included in the build. Here is an example of a CMake target_sources command which adds the NeuralNetwork.cpp file to the build for a target called fm-torchknob:

```
target_sources(fm-torchknob
    PRIVATE
    src/PluginEditor.cpp
    src/PluginProcessor.cpp
    src/NeuralNetwork.cpp)
```

You might add the header files to the list as well, if that is appropriate for your setup. I have seen people adding header files when working in Visual Studio Community as it makes them easier to find in the project. Now back to the NeuralNetwork.cpp file, here is an implementation for the NeuralNetwork constructor which takes two 64-bit integers for the input and output size and uses them to initialise a Linear layer:

```
NeuralNetwork::NeuralNetwork(int64_t _n_inputs, int64_t _n_outputs)
: n_inputs{_n_inputs}, n_outputs{_n_outputs}
{
    linear = register_module(
                "linear",
                torch::nn::Linear(n_inputs, n_outputs)
            );
}
```

You can see here that I am calling register_module, which is a function inherited from the torch Module class. This allows the Module parent class to keep track of the layers (modules) we are adding to the network. Later you might use the functions provided by Module to do various useful operations on the network as a whole, which is why you need to register the modules.

Now the rest of the functions – here is some placeholder code, for now, just to get it to compile:

```
std::vector<float> NeuralNetwork::forward(
             const std::vector<float>& inputs)
{
    std::vector<float> out = {0, 0};
    return out;
}

void NeuralNetwork::addTrainingData(
             std::vector<float>inputs,
             std::vector<float>outputs)
{

```

```
}

void NeuralNetwork::runTraining(int epochs)
{

}

torch::Tensor NeuralNetwork::forward(const torch::Tensor& input)
{
    torch::Tensor out = linear(input);
    return out;
}
```

To make sure compiling and linking are working, add a NeuralNetwork to the private area of the PluginEditor.h file:

```
NeuralNetwork nn{1, 2};
```

Then in the sliderValueChanged function, put this in the block that responds to the superknob:

```
// call forward on NeuralNetwork
std::vector<float> nn_outs = nn.forward(
    // pass it a vector containing the superknob value
    std::vector<float>{(float)superKnob.getValue()});

// use the return data to set the slider values
modDepthSlider.setValue(nn_outs[0]);
modIndexSlider.setValue(nn_outs[1]);
```

This code calls forward on our new NeuralNetwork class, passing it the superknob value and uses the output to set the values on the other two sliders. Build and run to check for any mistakes. Note that the public forward function on NeuralNetwork currently returns two zeroes – it does not actually use the neural network model to infer the slider values.

## 14.4 Unit testing

At this point, it might be worth setting up your CMake project to allow for lightweight unit testing. I do not know how fast your compile time is, but building a complete JUCE application every time I tweak the neural network code is slower than I would like on my system. You can create a simple program to test your NeuralNetwork class which will build and run faster, allowing for a faster development cycle. There are various formal ways to conduct unit testing with CMake, but here is my simple and informal method.

Add a new file to your src folder called test_nn.cpp. Put the following in that file:

```
#include "NeuralNetwork.h"
#include <iostream>

int main()
{
    NeuralNetwork nn{2, 2};
    std::cout << nn.forward({0.1, 0.5}) << std::endl;
    return 0;
}
```

As you can see, we are just creating a NeuralNetwork object and then calling forward on it, passing in some values. To add this to your CMake project, add the following lines to the end of CMakeLists.txt:

```
add_executable(test_nn
  src/NeuralNetwork.cpp
  src/test_nn.cpp)
target_link_libraries(test_nn "${TORCH_LIBRARIES}")
set_property(TARGET test_nn PROPERTY CXX_STANDARD 14)
```

When you re-run CMake to regenerate the project, you should have another executable target you can build. In VSCode, this is accessible from the dropdown menus for the build and run buttons. You can now choose if you want to do quick tests of the NeuralNetwork class by editing test_nn.cpp or if you want to test the full integration with the FM synthesizer. You can add a series of test functions to the test_nn.cpp file.

## 14.5 Implement NeuralNetwork's forward function

Next, you will replace the code in the NeuralNetwork.cpp file for the two forward functions with some proper code to pass data through the network. First, the public function:

```
std::vector<float> NeuralNetwork::forward(std::vector<float> inputs)
{
    // copy input data into a tensor
    torch::Tensor in_t = torch::empty({1, n_inputs});
    for (long i=0; i<n_inputs; ++i){
        in_t[0][i] = inputs[i];
    }
    // pass through the network:
    torch::Tensor out_t = forward(in_t);
    // initialise size to n_outputs
    std::vector<float> outputs(n_outputs);
    // copy output back out to a vector
    for (long i=0; i<n_outputs; ++i){
        outputs[i] = out_t[0][i].item<float>();
    }
    return outputs;
}
```

Some comments on the forward function:

1. Essentially, it makes a tensor for the input data, copies the vector data to the tensor, passes the tensor to the network, copies the output of the network back out to a vector and returns it.
2. The code is generalised, so it can deal with any number of inputs and any number of outputs.
3. It uses some dynamic memory allocation to create the tensors and vectors. This is definitely not the most efficient way to implement this code but this code will run at 'GUI' speed as opposed to audio speed so it does not need to be highly optimised yet.
4. A more efficient way would be to create the vectors and tensors as class data members and re-use them, but that would complicate the code, and I want to keep it as clear as possible for now.

Next, the private forward function:

```
torch::Tensor NeuralNetwork::forward(const torch::Tensor& input)
{
    torch::Tensor out = linear(input);
    return out;
}
```

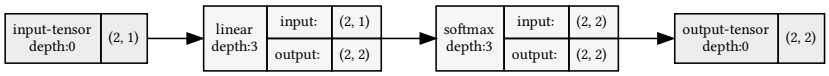


**FIGURE 14.2**
Basic architecture where a linear layer passes into a softmax layer. The numbers in the brackets indicate input and output shape. The linear layer input (2,1) goes from 1 value to 2 nodes, then output (2,2) goes from 2 nodes to 2 outputs.

Now try re-running the test_nn program a few times. You should see different outputs each time. Can you figure out why the outputs differ each time? Are these outputs in suitable ranges for the sliders?

## 14.6 Normalising the network output: softmax

The neural network outputs can be in all kinds of ranges, depending on the weights and biases in the network and the input. To push the outputs into more predictable ranges, libtorch provides a few options and softmax is one such option. Softmax takes a set of output values and modifies them so they add up to a total of 1. For the mathematically minded, it can be expressed as follows:

$$Softmax(x_i) = \frac{exp(x_i)}{\Sigma_j exp(x_j)} \tag{14.1}$$

In other words, the softmax value of output $x_i$ is the exponential of that output divided by the sum of the exponents of all outputs. Let's run through a calculation to make it really clear. Say we had two outputs with values 10.2 and 1.5. We take the exponent of those values:

$$exp(10.2) = 26903.1861 \tag{14.2}$$

$$exp(1.5) = 4.4817 \tag{14.3}$$

We sum those values:

$$exp(10.2) + exp(1.5) = 26907.6678 \tag{14.4}$$

Then to compute softmax of 10.2:

$$\frac{exp(10.2)}{exp(10.2) + exp(1.5)} = 0.9998 \tag{14.5}$$

and softmax of 1.5:

$$\frac{exp(1.5)}{exp(10.2) + exp(1.5)} = 0.0002 \tag{14.6}$$

Since the sum of the softmax of all outputs is 1, softmax is typically used to convert output layer values into probabilities. This is useful for classification, which is a common task for neural networks. For example, does a given image contain a dog or a cat? In classification tasks, the outputs represent the network's estimation of the probability of an input belonging to a given class, and softmax is a handy layer that converts outputs in any range to outputs in the range 0-1, totalling 1. So you can use softmax as a convenient processing layer to put the outputs in predictable ranges. With the NeuralNetwork class, it is fairly easy to add new layers. Here are the steps to add a new layer:

**Step 1:** Add a field of the appropriate type for your layer to the private section of the NeuralNetwork class in NeuralNetwork.h. For a softmax layer:

```
torch::nn::Softmax{nullptr};
```

**Step 2:** Call register_module in the constructor of NeuralNetwork:

```
softmax = register_module(
   "softmax", // name it
   torch::nn::Softmax(1) // configure it
);
```

Note that softmax takes one argument to its constructor, and a '1' is appropriate for our purposes. According to the PyTorch documentation, this constructor argument 'dim' is 'A dimension along which softmax will be computed (so every slice along dim will sum to 1).'

**Step 3:** Pass the output of the linear layer through the softmax layer in the NeuralNetwork's private forward function. Here is the new, complete forward function:

```
torch::Tensor NeuralNetwork::forward(const torch::Tensor& input)
{
    std::cout << "forward input " << input << std::endl;
    torch::Tensor out = linear(input);
    std::cout << "forward after linear " << out << std::endl;
    out = softmax(out);
    std::cout << "forward after softmax " << out << std::endl;
    return out;
}
```

Note that I have added some print statements to the code so we can check how the data flows through the network. You can see a visualization of the neural network architecture in figure 14.2.

## 14.7 Testing the simple model

Hopefully, you followed the instructions earlier and set up a simple test program as an additional executable in your CMake project. If you did, you could quickly try out the following test program to check on the output of your neural network:

```
#include "NeuralNetwork.h"
#include <iostream>

int main()
{
    NeuralNetwork nn{1, 2};
    for (float i=0;i<1; ++i){
        std::cout << nn.forward({i/10}) << std::endl;
    }
    return 0;
}
```

On my system, I see output as follows (slightly truncated):

```
forward input  0
[ CPUFloatType{1,1} ]
forward after linear  0.2885  0.4512
[ CPUFloatType{1,2} ]
forward after softmax  0.4594  0.5406
[ CPUFloatType{1,2} ]
0.459409 0.540591
...
forward input  0.4000
[ CPUFloatType{1,1} ]
forward after linear  0.5363  0.1527
[ CPUFloatType{1,2} ]
forward after softmax  0.5947  0.4053
[ CPUFloatType{1,2} ]
0.594736 0.405264
----
forward input  0.9000
[ CPUFloatType{1,1} ]
forward after linear  0.8460 -0.2205
[ CPUFloatType{1,2} ]
forward after softmax  0.7439  0.2561
[ CPUFloatType{1,2} ]
0.743922 0.256078
```

You can now return to your JUCE synthesizer and test the new neural network. Remember that the network outputs are in the range [0..1], so you should scale them into the correct ranges for the sliders.

## 14.8 Progress check

At this point, you should have a working JUCE/ libtorch plugin project. You should have a neural network class and be able to take data from a JUCE slider and pass it through the neural network. You should be able to pass the output of a linear layer in the network through a softmax layer to normalise it. You should be able to use the normalised neural network output to move two other sliders. You should have a test program that appears as a separate target in your CMake project.

# 15

# *Training the torchknob*

The chapter begins by explaining how to train the neural network that powers the torchknob. First, the focus is on preparing the user interface for training by adding data input knobs and function buttons. Then, the process of gathering a training dataset by capturing input and output values from the user interface is demonstrated. After that, the chapter describes how the training data is converted to tensor format and stored. The implementation of the training loop is then discussed, wherein a Stochastic Gradient Descent optimiser is used to update the neural network parameters. Additionally, the chapter reconsiders using the softmax layer for output normalization. It suggests replacing it with a sigmoid activation function, the output normalization method utilised in the original Wekinator and learner.js systems.

## 15.1 Preparing the user interface for training

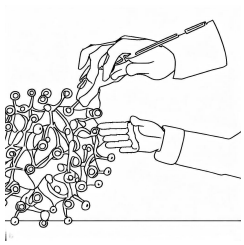

**FIGURE 15.1**
Interactive machine learning provides more intuitive training for neural networks.

You need a dataset containing example input and output pairs to train a neural network. The idea is that you will use this data to train the network to produce the correct output when presented with the correct input. Once it has learned the mapping it can infer new outputs given inputs it has not necessarily seen before. To put this in the context of the FM synthesizer, you are saying: 'When the superknob is here, I want the other sliders to be here'. You usually need several examples to train the network. You will probably be aware that giant datasets containing millions or billions of examples are needed to train large networks. For our purposes, though, we can achieve decent functionality with just a few examples. For those more versed in neural networks, we will exploit over-fitting as we are just looking for interesting regressions between the data points.

We will gather the training data using an iterated 'learn

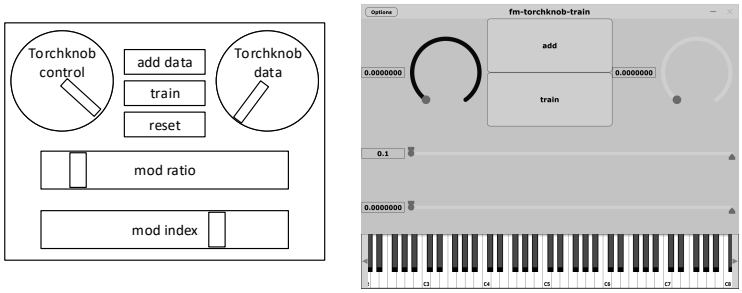


**FIGURE 15.2**
User interface mockup for trainable superknob system (left). We have an additional knob to specify training input without triggering the movement of the sliders. Actual user interface prototype (right).

by example' approach, sometimes referred to as *interactive* machine learning. We need to change the user interface to have the required elements to allow the user to interactively present examples to the training system. Figure 15.2 shows a wireframe mockup of the interface. It has the following additional elements:

1. A training knob to allow you to specify the inputs for training without the sliders moving
2. An add button to add a training example to the training dataset
3. A train button to trigger training

Your task is to implement the code necessary in the PluginEditor class, so it has the user interface elements shown in 15.2. Go ahead and add the UI widgets to your PluginEditor.h and cpp files. Ensure you add the PluginEditor as a listener to the buttons and sliders. You will implement the actual listener code in the next section. You do not need to stick to the exact design and layout shown – there are other ways to implement the UI. You could work out a way to have a single knob and some sort of 'training' mode. Then you would have a toggle to switch between train and infer mode. That requires somewhat more complex application logic, so I leave it to you to decide if you want to implement that. If you want to change the colours of the widgets, here is some example code wherein I change the colour of a UI element called superKnobTrain:

```
superKnobTrain.setColour(
  // which part of the slider are we changing?
  juce::Slider::ColourIds::rotarySliderOutlineColourId,
  // what is the new colour?
  juce::Colours::aliceblue
);
```

## 15.2 Gather a dataset

Now you have the user interface up and running, you can implement the code to add training examples for the neural network. You need to gather data from the user interface and then pass it to the NeuralNetwork class, which stores it. Locate the buttonClicked function in PluginEditor.cpp. Write some code that responds to the add button. In my case, the add button is called 'addExampleButton':

```
if (btn == &addExampleButton){
    // read the value from the training control:
    float in = (float) superKnobTrain.getValue();
    // read the current values for the synth parameter sliders:
    float out1 = (float)modDepthSlider.getValue() /
                        modDepthSlider.getMaximum();
    float out2 = (float)modIndexSlider.getValue() /
                    modIndexSlider.getMaximum();
    // send the input-output pair to the neural network
    // so it can be used in training later
    nn.addTrainingData({in},{out1, out2});
}
```

## 15.3 Implement training data storage

Now you have completed the UI side, it is time to implement the training data storage in the NeuralNetwork class. Add these data members to the private section of NeuralNetwork.h:

```
std::vector<torch::Tensor> trainInputs;
std::vector<torch::Tensor> trainOutputs;
```

You can see that these variables store vectors of tensors. I will admit that there are more 'torchy' ways to manage a dataset, but it is the simplest for now, without digging into the torch::datasets classes. Then you need to change addTrainingData so it converts the incoming vectors of floats into tensors and stores them into those tensor vectors:

```
// converting the input vector 'inputs'
// to a tensor
torch::Tensor inT = torch::from_blob(
    // get access to the raw memory storing
    // the vector of floats
    (float*)(inputs.data()),
    inputs.size()
    // then make a copy of it
```

```
    ).clone();
// same process for the outputs
torch::Tensor outT = torch::from_blob(
    (float*)(outputs.data()),
    outputs.size()
    ).clone();
trainInputs.push_back(inT);
trainOutputs.push_back(outT);
```

The from_blob function allows you to create a tensor directly from the data in the vector which is a bit more efficient on lines of code and memory allocation than manually coping the vector's data into the tensor. The clone function copies the data from the vector to ensure that data belongs to the trainInputs vectors (otherwise, the memory storing the data can be reassigned). As before, drop down to your test_nn.cpp file to quickly test that things are working as expected. Use printouts to verify the data from the sliders ends up in the training tensors. The test file is handy because JUCE's DBG function cannot handle printing tensors. When I was debugging this code, it helped me to verify data coming into the network.

Another useful technique for testing and managing errors is assertion. To verify that the data being sent to the addTrainingData function has the correct shape, you can add the following two lines to the addTrainingData function in NeuralNetwork.cpp:

```
assert(inputs.size() == (unsigned long) n_inputs);
assert(outputs.size() == (unsigned long) n_outputs);
```

That code will trigger a crash if the vectors have the wrong size. Better than limping on and crashing out later with an intractable bug. Note that I had to cast the n_inputs variable from its initial type of int64_t into an unsigned long to avoid a compiler warning. Tensor sizes are int64_t but vector sizes are unsigned longs.

## 15.4 Write the runTraining code

The final stage is to implement the runTraining function, which will train the neural network using an error metric and an optimiser. The components at play here are similar to those from the linear regressor example you worked on before. They are just reorganised into the NeuralNetwork class. First, the optimiser – add a data member to the private section of NeuralNetwork.h

```
// unique ptr so we can initialise it
// in the constructor after building the model
std::unique_ptr<torch::optim::SGD> optimiser;
```

Note that I am using a unique_ptr here. This allows me to decide which parameters the optimiser receives from its constructor after I have created the neural network model. In fact, I only know what the parameters are once I create the model by calling the register_module function in the neural network's constructor. So I have to dynamically create the SGD. I cannot use this style as I did for the network layers:

```
torch::optim::SGD optimiser{nullptr};
```

SGD, unfortunately, does not have a constructor with that signature. If you are not very experienced with C++, you may not be familiar with unique_ptr or even pointers. What you need to know now is that pointers were inherited by C++ from the C language, and they allow programs to request memory from the operating system when they are running, as opposed to requesting all needed memory at the start. If you use a pointer to allocate memory, you must also return that memory to the system. Otherwise, you have a memory leak, where your program takes memory but does not return it. Pointers are notorious for striking fear and confusion into the hearts of novice C++ programmers, and they can cause all kinds of problems. Unique_ptr and other so-called 'smart pointers' were invented in a later version of C++ to help solve some of these problems. Smart pointers automatically free the memory they use once they go out of scope. In other words, they behave more like regular variables with the added capability of runtime memory allocation. You will see smart pointers such as unique_ptr and shared_ptr popping up in many of the JUCE example applications that come with the JUCE developer kit.

I am using a smart pointer here as it allows me to create an SGD variable without assigning an actual SGD object to it. This is helpful as I can only create an SGD object if I know the parameters for the neural network it will optimise. So with my smart pointer, I can create the object after I have created the neural network model in the NeuralNetwork class constructor. Here is some code that uses the std::make_unique function to create an SGD object wrapped in a unique_ptr. Put the code at the end of the NeuralNetwork constructor in NeuralNetwork.cpp:

```
optimiser = std::make_unique<torch::optim::SGD>(
    this->parameters(), .01);// params, learning rate
```

Note how I call the parameters function on 'this'. Remember that our NeuralNetwork class inherits from the torch Module class. This means it inherits a set of functions from Module. One of those functions is 'parameters', and it returns the parameters of the registered modules in a form that the optimiser can understand. 'this' refers to the current NeuralNetwork object. I could just have called 'parameters' without 'this', but I prefer this more explicit syntax as it tells you parameters is a function inside the current object, not something global or otherwise in the namespace somewhere.

Next, the actual runTraining function. This is where things get somewhat

complex as we first convert the training data into a large tensor, then use it in combination with the optimiser to improve the weights:

```
//Push inputs to one big tensor
torch::Tensor inputs = torch::cat(trainInputs)
.reshape({(signed long) trainInputs.size(), trainInputs[0].sizes()
    [0]});
//Push outputs to one big tensor
torch::Tensor outputs = torch::cat(trainOutputs)
.reshape({(signed long) trainOutputs.size(), trainOutputs[0].sizes()
    [0]});
//Run the training loop
for (int i=0;i<epochs; ++i){
    //Clear out the optimizer
    this->optimiser->zero_grad();
    auto loss_result = torch::mse_loss(forward(inputs), outputs);
    float loss = loss_result.item<float>();
    std::cout << "iter: " << i << "loss " << loss << std::endl;
    // Backward pass
    loss_result.backward();
    // Apply gradients
    this->optimiser->step();
}
```

Again, I recommend dropping to your test_nn.cpp file and trying a simple test program. Here is an example of a simple testing program:

```
int main()
{
    NeuralNetwork nn{1, 2};
    for (float i=0;i<10; ++i){
        nn.addTrainingData({i/10}, {i/5, i/3});
    }
    nn.runTraining(10);
    return 0;
}
```

Satisfy yourself that the data is flowing around the system correctly with prints and tests in the test function, then return to the JUCE application and verify that it also works correctly.

## 15.5 Experiment with training

Figure 15.3 shows an example experiment you can carry out with the torchknob-controlled synthesizer. Try training the network for different numbers of epochs. Can you achieve a satisfactory result where the model moves the sliders to the correct positions you trained it on?

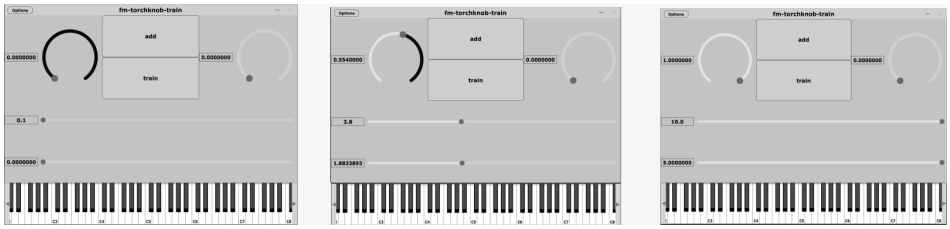


**FIGURE 15.3**
Example of an experiment you can carry out. First, set the training slider to its lowest value, the same for the modulation controls. Add a training point. Then move to the middle positions, and add a training point. Finally, move to the highest positions, and add a training point.

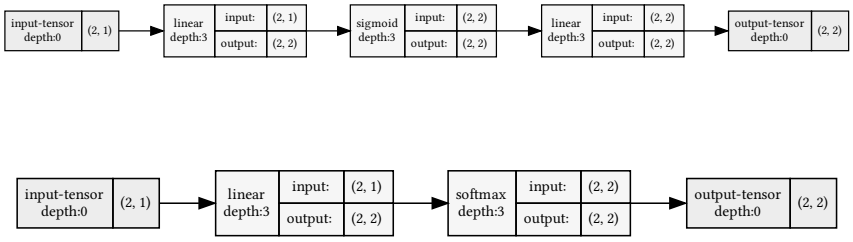


**FIGURE 15.4**
The learner.js/ Wekinator regression architecture (top). The simpler architecture we used previously (bottom).

## 15.6 The ideal model

If you carried out the experiment as suggested, you probably found that the basic model with one linear layer followed by a softmax did not manage to learn the mapping between inputs and outputs very well – the torchknob did not move the sliders to the correct positions. Do not lose hope – we just need to adjust the model! Sorry about that – I should have given you the ideal model at the start. But you learned more about training and softmax as a result, which might come in useful later.

I mentioned earlier that Louis McCallum created an implementation of the

Wekinator in Javascript. It is called learner.js. If you look carefully at the learner.js code on github[1], you can find the specification of the model there. It is written in tensorflow.js, but it is possible to recreate the exact model in libtorch. The model has two fully connected Linear layers with a Sigmoid activation function between them. Figure 15.4 shows a visualization of the model. In libtorch terms, you need three variables in your NeuralNetwork class:

```
torch::nn::Linear linear1{nullptr};
torch::nn::Sigmoid sig1{nullptr};
torch::nn::Linear linear2{nullptr};
```

Set them up in the constructor like this:

```
linear1 = register_module(
    "linear1",
    torch::nn::Linear(n_inputs, n_outputs)
);
sig1 = register_module("sig1", torch::nn::Sigmoid());
linear2 = register_module(
    "linear2",
    torch::nn::Linear(n_outputs, n_outputs)
);
```

Then process the inputs like this:

```
torch::Tensor out = linear1(input);
out = sig1(out);
out = linear2(out);
```

Note that there is no activation function after the second linear layer, or even a softmax layer. But since it is well tested within the Wekinator systems, this model should be able to learn the mappings much more effectively than the simpler linear/ softmax model can. Run some tests with your test_nn.cpp setup. Does it achieve lower errors than the linear/softmax network? If you want to dig more into the experimental process here, you can create two NeuralNetwork classes, one with the original model and the other with this new one. Compare and contrast using test code. Being able to compare the performance of different neural network models on the same dataset is an important skill to learn.

In order to avoid offending experienced machine learning engineers, I should note that this method of evaluating neural networks by measuring their error on the training set is not the standard way to do things. Typically, you would split the dataset into a training set, a validation set and maybe even a test set. The training is carried out using the training set, you use the validation set error to monitor the progress of training to avoid over-fitting, and you use the test set as an additional resource to validate the training. But here we have a very small training set which might only consist of a few items. So it is not possible to split the data like that. This workflow is designed to allow for simple, interactive machine learning with a

[1] https://github.com/Louismac/learnerjs/blob/main/libs/learner.v.0.5.js

small amount of data. In other examples in the book, I will demonstrate the full and proper process of training a neural network with a non-interactive workflow.

## 15.7 The ideal training loop

Now you have the ideal model for the task, you need a smarter training function. Smarter in that it has a few more measures in there to decide when the training is complete. Right now, the training loop just runs for the specified number of epochs. But how do you know if it is trained enough? One approach would be to observe the change in error after each epoch. If the change becomes very small, it is trained enough. Try this in your training loop:

```
float loss{0}, pLoss{1000}, dLoss{1000};
for (int i=0;i<epochs; ++i){
    //Clear out the optimiser
    this->optimiser->zero_grad();
    auto loss_result = torch::mse_loss(forward(inputs), outputs);
    loss = loss_result.item<float>();
    loss_result.backward();
    this->optimiser->step();
    //Now compute the change in loss
    // and exit if it is too low
    dLoss = pLoss - loss;
    pLoss = loss; // prev loss
    if (i > 0){// only after first iteration
        if (dLoss < 0.00001) {
            std::cout << "dLoss low, exit on ep " << i << std::endl;
            break;
        }
    }
}
```

How many epochs does it take you to train in the test program? My test program takes around 5–6,000 epochs, and the main torchknob program takes about the same number. However, this depends on how many data points you have to feed it. Try adding more layers to the network or increasing the 'hidden layer' size. Adding a reset button that deletes the training data and randomises the network weights would be helpful.

## 15.8 Next steps

Here are few more experiments you can try:

1. Try passing the NN conflicting values. Observe that it cannot work miracles – if you put two data points in two different places, it will just sit in the middle to minimise the error.
2. Make a more complex synthesis algorithm that uses more parameters.
3. Make a custom or more sophisticated GUI element that generates more control inputs, e.g. a 2D trackpad-type controller.

## 15.9 Progress check

At this point you should be able to train a neural network to learn a regression through parameter space. You should be able to do so by interactively creating a dataset of input-output pairs using a user interface. You should have experimented with different training loops and different neural network models.

# 16

# *Plugin meta-controller*

This chapter covers the steps involved in adapting the torchknob FM synthesizer into a plugin host. This means we can move from the torchknob concept, wherein a neural network learns to control a fixed synthesizer's parameters, to the meta-controller concept, wherein a neural network can control any plugin synthesizer available. The steps covered are removing the FM synthesizer UI and DSP code, adding code to load a VST plugin synthesizer, and hooking the plugin into the processBlock function so it can generate audio and process MIDI.

## 16.1 Hosting plugins

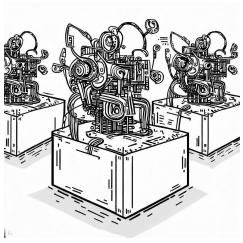

**FIGURE 16.1**
Hosting plugins allows you to control more advanced synthesizers.

The first step is to adapt the torchknob FM synthesiser plugin into a plugin host. The concept is that your plugin will use an external plugin to generate its audio instead of the FM synthesiser DSP code.

Make a copy of the FM torchknob you created in the previous sections. A reference implementation of this project is described in section 39.3.4. You can start with that reference implementation if you want to follow these instructions as closely as possible. Or, if you want a shortcut, you can skip to section 16.2 and start with the clean project 39.3.5.

Remember to change all occurrences of the project name in the CMakeLists.txt file and to change the PLUGIN_CODE. In my case, working from the FM synthesiser reference project, I carried out a 'find and replace' replacing 'fm-torchknob-train' with 'plugin-host' and then changed the code from cbc3 to dbc1.

Next, you need to enable plugin hosting capabilities in your project. Locate the 'target_compile_definitions' block in the CMakeLists.txt file. Set the property 'JUCE_PLUGINHOST_VST3' to 1, e.g.

```
JUCE_PLUGINHOST_VST3=1
```

That will enable plugin hosting capability in your application. Now generate the project, build and run to verify things are ready.

### 16.1.1 Remove the FM synthesis UI code and DSP

Starting from the reference project, the UI controls to remove are the two sliders for modulation depth and modulation index and the toggle button for the constant envelope mode. My approach here is to comment out the UI objects in the PluginEditor.h file:

```
// juce::ToggleButton envToggle;
// juce::Slider modIndexSlider;
// juce::Slider modDepthSlider;
```

Then in the PluginEditor.cpp, comment out any code that uses those variables, i.e. the relevant lines in the constructor, the resized function, the sliderValueChanged function and the buttonClicked function. Once you have done this, build and run to verify it still builds.

Next, for the code in PluginProcessor. Remove all synthesis-related variables from the private section of PluginProcessor.h except:

```
juce::MidiBuffer midiToProcess;
```

Remove all synthesis-related functions in PluginProcessor.h, essentially, anything that is not an overridden function. Leave this one, though:

```
void addMidi(juce::MidiMessage msg, int sampleOffset);
```

We will need that to make the piano keyboard widget on the UI work. Now go into PluginProcessor.cpp and remove any lines referring to those variables and definitions of functions you have removed. Your IDE will probably go crazy with red lines and warnings during this process. Finally, you can strip back the processBlock function to something like this:

```
// add collected MIDI to the function's MIDI input
if (midiToProcess.getNumEvents() > 0){
    midiMessages.addEvents(midiToProcess, midiToProcess.
    getFirstEventTime(), midiToProcess.getLastEventTime()+1, 0);
    midiToProcess.clear();
}
//Just clear the buffers
juce::ScopedNoDenormals noDenormals;
auto totalNumInputChannels  = getTotalNumInputChannels();
auto totalNumOutputChannels = getTotalNumOutputChannels();
for (auto i = totalNumInputChannels; i < totalNumOutputChannels; ++i)
    buffer.clear (i, 0, buffer.getNumSamples());
```

For completeness, I also renamed the two classes: FMPluginProcessor becomes PluginHostProcessor, and FMPluginEditor becomes PluginHostProcessor.

If it all goes terribly wrong, you can refer to the project 39.3.5.

## 16.2 Load a plugin: hard coded location

At this point, you should have a stripped-back plugin project which includes the NeuralNetwork class code and the training control widgets.

We shall start by loading a plugin from a hard-coded location on the file system. Declare a public function in PluginProcessor.h:

```
void loadPlugin(const juce::File& pluginFile);
```

Const tells the compiler we will not make any changes to the pluginFile variable (the variable, not the file) and the & tells the compiler not to make a copy of it.

Add these variables to the PluginProcessor.h private data members:

```
juce::AudioPluginFormatManager pluginFormatManager;
juce::KnownPluginList knownPluginList;
juce::OwnedArray<juce::PluginDescription> pluginDescriptions;
int vstFormatInd;
std::unique_ptr<juce::AudioPluginInstance> pluginInstance;
```

1. AudioPluginFormatManager: provides functionality to load different plugin formats
2. KnownPluginList: not so much a list as a list manager. Can load and save plugin descriptions
3. PluginDescription array: the KnownPluginList manages this list of known plugins
4. AudioPluginInstance: this will point to the plugin once we load it

The class names for the JUCE API are confusing here since KnownPluginList is not a list but a manager of a list. The actual list is the OwnedArray of PluginDescriptions.

In the constructor, initialise the manager and locate the VST3 plugin in its list of known formats. We will need to use that VST3 format index later, and this is a reasonable time to locate it.

```
pluginFormatManager.addDefaultFormats();
int currInd{0}, vstInd{0};
for (const juce::AudioPluginFormat* f : pluginFormatManager.
    getFormats()){
    if (f->getName() == "VST3"){
        vstFormatInd = currInd;
        break;
    }
    currInd++;
}
```

Now everything is set for the loadPlugin function. Here is a minimal implementation of loading a plugin:

```
// do not call processBlock when loading a plugin
suspendProcessing(true);

// remove any previously read descs
// so we can just use index 0 to find our desired on layer
pluginDescriptions.clear();
bool added = knownPluginList.scanAndAddFile(
                        pluginFile.getFullPathName(),
                        true,
                        pluginDescriptions,
                        *pluginFormatManager.getFormat(vstFormatInd));

juce::String errorMsg{""};
pluginInstance = pluginFormatManager.createPluginInstance(
    *pluginDescriptions[0], // 0 since we emptied the list at the top
    getSampleRate(), getBlockSize(), errorMsg);

// get the plugin ready to play
pluginInstance->enableAllBuses();
pluginInstance->prepareToPlay(getSampleRate(), getBlockSize());
// re-enable processBlock
suspendProcessing(false);
```

Some comments on that code:

1. suspendProcessing prevents any audio processing until the plugin is ready
2. scanAndAddFile checks if the sent file can be loaded as a plugin
3. added is true if we can load this plugin
4. createPluginInstance actually loads a plugin into memory
5. createPluginInstance writes its result into errorMsg
6. enableAllBuses makes sure the plugin is properly initialised

At this point, you should add some checks to the loadPlugin function; for example, if the variable ‘added’ ends up as false, you cannot load the plugin. If

the variable 'errorMsg' is empty after you call createPluginInstance, the plugin should be ready to use.

To test out loadPlugin, locate a plugin on your system. There are default locations for plugins and these vary depending on your operating system. You can query the default location using the JUCE VST3PluginFormat's getDefaultLocationsToSearch function[1]. Find a VST3 plugin using your operating system's file browser or terminal and get the full file path. Then call loadPlugin in AudioProcessor.cpp's prepareToPlay function, as that is called when the audio system is ready to go. If you call it somewhere else, e.g. the constructor, you might not know the sample rate or the block size yet, and you need those to create the plugin. Your plugin should load, and the appropriate messages you coded earlier should appear.

## 16.3 Use the plugin in processBlock

Now you can load a plugin; it is time to use it to generate audio in response to MIDI input. You will need a VSTi plugin, in other words, a synthesiser, as effects units do not generate audio without an input (generally speaking). I recommend you test the system using one of the plugins you have made, e.g. the FM superknob one. I have a few different test plugins, some generating a continuous tone.

Verify you can load a synthesiser plugin and then put the following in processBlock (the only code in processBlock):

```
// process any MIDI captured from the on-screen
// piano keyboard widget
if (midiToProcess.getNumEvents() > 0){
    midiMessages.addEvents(midiToProcess, midiToProcess.
    getFirstEventTime(), midiToProcess.getLastEventTime()+1, 0);
    midiToProcess.clear();
}
// call processBlock on the hosted plugin
if (pluginInstance){// we have a plugin loaded
    pluginInstance->processBlock(buffer, midiMessages);
}
```

This code is a good start – you can now host a plugin and hear its output. Code example 39.3.6 in the repo guide provides a fully working example.

[1] https://docs.juce.com/develop/classVST3PluginFormat.html

## 16.4 Progress check

At this point you should have a plugin that can load and host another plugin. You should be able to call processBlock on the hosted plugin and it should generate an audio signal if you pass it MIDI note data.

# 17

# *Placing plugins in an AudioProcessGraph structure*

This chapter describes the process of integrating plugins into an AudioProcessGraph structure. AudioProcessorGraph is a built-in class in the JUCE library which allows the creation of a graph with multiple plugins connected together, managing MIDI and other complexities. With this graph, it is possible to host multiple plugins simultaneously. In fact, it is the underlying technology for the AudioPluginHost plugin test environment. The chapter goes through all the steps involved in converting the existing plugin controller to work with the AudioProcessGraph. These steps are adding the required fields for the graph and creating MIDI and audio input/output nodes, initialising the graph, wrapping the plugin in a graph node and using the graph in the processBlock function.

## 17.1 What is an AudioProcessorGraph?

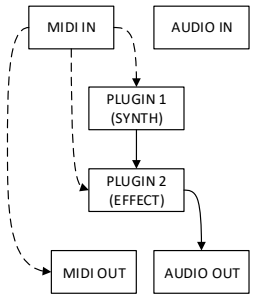


**FIGURE 17.1**
Wiring plugins together with a processor graph.

Before we get started, I would like to say that there are some quite large chunks of source code in this section. They are there for completeness so you can see the full implementation of graph-based audio processing in JUCE. But I would not expect you to type in all of this code from the book. You can refer to example 39.3.7 in the repo guide which has a fully working implementation of the plugin graph.

AudioProcessorGraph is a class in the JUCE API that allows you to treat audio and MIDI processing units like nodes in a graph. If you have used the AudioPluginHost plugin testing environment, you have already seen a fully developed example of what is possible with the AudioProcessorGraph. If you have not seen it, I recommend you check it out

now. You can find AudioPluginHost in the extras folder of the JUCE distribution. Build it and run it. Figure 17.1 presents a simple example of a graph you can create with the AudioProcessorGraph. In that example are MIDI in and out nodes and audio in and out nodes. Two plugin nodes are embedded in the graph: a synthesizer and an effects unit.

Both plugins receive MIDI from the MIDI-in node allowing them to be controlled. The MIDI-in is also wired to the MIDI-out – a kind of MIDI-thru for those familiar with outboard synthesizers. The audio-in is not wired to anything. Audio from the synthesizer node passes to the effects unit node and then to the audio-out.

## 17.2 Add fields for AudioProcessorGraph

The first step is to add some new variables to the PluginProcessor to allow for the AudioProcessorGraph. You will notice that in the list of variables below there are AudioPluginInstance variables and Node :: Ptr variables. Conceptually, the Nodes are wrappers around the AudioPluginInstances. You create the AudioPluginInstance variables, then pass them to the Nodes which manage them, There are a few programming techniques going on here to make things work in a memory leak-resistant manner. Let's first take a look at the variables, which you should add to the private section of the PluginProcessor.h file:

```
//The graph
std::unique_ptr<juce::AudioProcessorGraph> audioProcGraph;

// graph components: processors
std::unique_ptr<juce::AudioPluginInstance>  inputProc;
std::unique_ptr<juce::AudioPluginInstance>  outputProc;
std::unique_ptr<juce::AudioPluginInstance>  midiInputProc;
std::unique_ptr<juce::AudioPluginInstance>  midiOutputProc;
// graph components: nodes that wrap the processors
juce::AudioProcessorGraph::Node::Ptr  pluginNode;
juce::AudioProcessorGraph::Node::Ptr  inputNode;
juce::AudioProcessorGraph::Node::Ptr  outputNode;
juce::AudioProcessorGraph::Node::Ptr  midiInNode;
juce::AudioProcessorGraph::Node::Ptr  midiOutNode;
```

The AudioPluginInstance variables are unique_ptrs and the graph nodes are Node::Ptrs. More on that shortly.

## 17.3 Initialise the graph

To initialise the graph, run the following in the constructor for PluginProcessor.h:

```
//Create basic processors for the graph
inputProc =
    std::make_unique
    <juce::AudioProcessorGraph::AudioGraphIOProcessor>
    (juce::AudioProcessorGraph::AudioGraphIOProcessor::audioInputNode
    );

outputProc =
    std::make_unique
    <juce::AudioProcessorGraph::AudioGraphIOProcessor>
    (juce::AudioProcessorGraph::AudioGraphIOProcessor::
    audioOutputNode);

midiInputProc =
    std::make_unique
    <juce::AudioProcessorGraph::AudioGraphIOProcessor>
    (juce::AudioProcessorGraph::AudioGraphIOProcessor::midiInputNode)
    ;

midiOutputProc =
    std::make_unique
    <juce::AudioProcessorGraph::AudioGraphIOProcessor>
    (juce::AudioProcessorGraph::AudioGraphIOProcessor::midiOutputNode
    );

audioProcGraph->enableAllBuses();
inputProc->enableAllBuses();
outputProc->enableAllBuses();

// convert the io processors into nodes
inputNode = audioProcGraph->addNode(std::move(inputProc));
outputNode = audioProcGraph->addNode(std::move(outputProc));
midiInNode = audioProcGraph->addNode(std::move(midiInputProc));
midiOutNode = audioProcGraph->addNode(std::move(midiOutputProc));
```

The code created a set of unique_ptr types, then called std::move to hand over the unique_ptrs to the nodes. We are getting into some gnarly C++ smart pointer territory here. I mentioned earlier that smart pointers manage dynamically allocated memory to ensure it is returned to the operating system when it is no longer needed. Unique_ptrs do this by only allowing one part (or one scope) of the program to access the unique_ptr at any time. Here, only the class we defined the variables in can access them. They cannot be passed around. If you want another part of the program to access a unique_ptr, you can pass it over with the move function. That is happening here: we create the unique_ptrs, then hand them over

to the node objects, which take over managing them. After calling move, this part of the program can no longer access the unique_ptr variables.

Read the code carefully – can you spot any variables we need to initialise? Did you spot that audioProcGraph still needs to be initialised? Add this to the initialiser list on the constructor:

```
    // e.g. just before #endif
, audioProcGraph{new juce::AudioProcessorGraph()}
```

To complete the discussion of unique_ptr, can you see the two ways we have instantiated the data stored in unique_ptr? First, we instantiated the AudioPluginInstance variables, such as midiOutputProc, by calling std::make_unique. Then, we instantiated the AudioProcessorGraph using an initialiser list. In other words, we called the constructor of AudioProcessorGraph's unique_ptr and passed it a pointer to an AudioProcessorGraph. Sheesh! The following short program illustrates these two ways of instantiating unique_ptrs, in case you re-encounter them:

```
#include <memory>

class Test {
    public:
    Test(){}
};

int main(){
    // method 1: instantiate by passing a 'real pointer'
    // to the constructor
    std::unique_ptr<Test> myTest{new Test()};
    // method 2: instantiate by calling make_unique:
    std::unique_ptr<Test> myTest2;
    myTest2 = std::make_unique<Test>();
}
```

## 17.4 Add the plugin to the graph

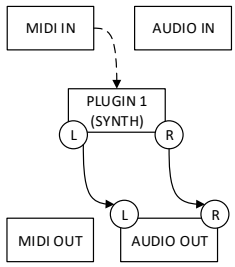


**FIGURE 17.2**
The graph you will create.

To get the plugin running properly inside the graph, you must adjust the loadPlugin function slightly. Just remove these two lines from loadPlugin:

```
pluginInstance->enableAllBuses();
pluginInstance->prepareToPlay(
      getSampleRate(),
      getBlockSize()
);
```

Now add a function with the following signature to the private section of PluginProcessor.h. This function will wrap the code that does the work of wiring the plugin into the graph as shown in figure 17.2. You can see in the figure that the MIDI input is wired to the plugin and the plugin is wired to the audio output. Here is the signature:

```
void addPluginToGraph();
```

And an implementation in PluginProcessor.cpp, firstly,

```
void PluginHostProcessor::addPluginToGraph()
{
    if (pluginNode) {
        audioProcGraph->removeNode(pluginNode);
    }
    pluginInstance->enableAllBuses();

    pluginNode = audioProcGraph->addNode (std::move (pluginInstance)
    );

}
```

Note how I used the pointer syntax to call the addNode function: instead of audioProcGraph.addNode it is audioProcGraph− >addNode. So far so good − we have a node sitting in the graph. Now connect the node to the appropriate input and output nodes by adding this code to the end of the addPluginToGraph function:

```
// connect the node to the output??
audioProcGraph->addConnection({
    {pluginNode->nodeID, 0},
    {outputNode->nodeID, 0}});
//This will crash if it's mono ... check if you want!
audioProcGraph->addConnection({
    {pluginNode->nodeID, 1},
```

```
    {outputNode->nodeID, 1}});
//Hook up midi in
audioProcGraph->addConnection({
    {midiInNode->nodeID,
     juce::AudioProcessorGraph::midiChannelIndex},
    {pluginNode->nodeID,
    juce::AudioProcessorGraph::midiChannelIndex}});

pluginNode->getProcessor()->prepareToPlay(getSampleRate(),
    getBlockSize());
```

Then call it from loadPlugin, before you 'unsuspend' the audio:

```
...
pluginInstance = pluginFormatManager.createPluginInstance(
    *pluginDescriptions[0],
    getSampleRate(), getBlockSize(), errorMsg);
if (errorMsg == "") {  DBG("Plugin probably loaded");}
else {DBG("Plugin failed to load " << errorMsg); return;}

// then:
addPluginToGraph();
suspendProcessing(false);
// end of loadPlugin
```

## 17.5 Call the graph in processBlock

The final step in moving to the graph model is to call processBlock on the graph in your host's processBlock function. This is very similar to how you did it with the pluginInstance except instead of calling processBlock on pluginInstance you call processBlock on audioProcGraph. Edit the processBlock function in AudioProcessor.cpp:

```
... midi stuff.

if (pluginNode){
    audioProcGraph->processBlock(buffer, midiMessages);
}
```

You should now be able to load the test plugin and hear it via the graph.

## 17.6 Load any plugin

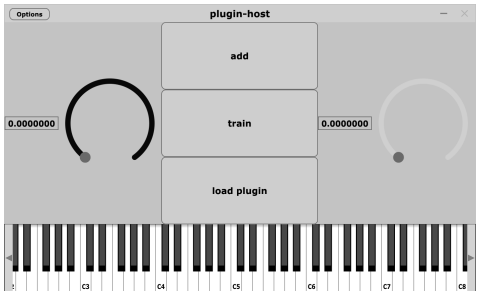


**FIGURE 17.3**
User interface for the basic host with the load plugin button added.

Now you will add a new button to the plugin interface that lets you load any plugin. Add a button to PluginEditor.h in the private section and follow the usual steps to make it appear: addAndMakeVisible, addListener, and update the resized function. You should end up with a user interface like figure 17.3. The new button should call the buttonClicked callback on the PluginEditor. But how to connect the button to the loadPlugin function? We need to trigger a file browsing dialogue, then collect the resulting filename and pass it to loadPlugin. The following code will show a file dialogue. Put it in the buttonClicked callback when the button is the load plugin button:

```
auto fileChooserFlags = juce::FileBrowserComponent::openMode |
                        juce::FileBrowserComponent::canSelectFiles;

fChooser.launchAsync(fileChooserFlags,
                     [this](const juce::FileChooser& chooser){
    DBG("you chose a plugin file " << chooser.getResult().
    getFullPathName());
    this->audioProcessor.loadPlugin(chooser.getResult());
});
```

To make it work, you need to add a private variable to the PluginEditor.h file:

```
juce::FileChooser fChooser{"Select a plugin."};
```

The code uses an anonymous function as an asynchronous callback. If you are familiar with Javascript, this pattern is familiar to you. The idea is that the GUI thread is not interrupted by the user browsing for a file, as the file browser happens in a separate thread. Once the user has selected a file, the code prints out the file path and then attempts to load it using the AudioProcessor's loadPlugin function. The fileChooseFlags specify that the user can select files. The problem is that plugins do not present themselves as files on all platforms. On some platforms they present themselves as directories.

If you want your application to be cross-platform, you need to specify different flags for different platforms. MacOS and Linux plugins are directories. Windows

plugins are files. So you need to set the flags using some JUCE macros to check which platform you are on.

```
# ifdef JUCE_LINUX
auto fileChooserFlags = juce::FileBrowserComponent::openMode |
                juce::FileBrowserComponent::canSelectDirectories;

# endif
# ifdef JUCE_MAC
auto fileChooserFlags = juce::FileBrowserComponent::openMode |
                juce::FileBrowserComponent::canSelectFiles;
# endif
# ifdef JUCE_WINDOWS
auto fileChooserFlags = juce::FileBrowserComponent::openMode |
                juce::FileBrowserComponent::canSelectFiles;
# endif
```

## 17.7 Progress check

At this point you should be able to load any plugin and play it using the piano control on the host application. Code example 39.3.7 in the repo guide provides a fully working example of the above.

# 18

# *Show a plugin's user interface*

The final functional change we will make to our plugin host before we switch back to working on the machine learning side is to display the user interface for the plugin. This chapter explains how to display the user interface for any plugin. One of the features of the various plugin APIs (VST3, AudioUnit, etc.) is that they allow the plugin to have a custom user interface. This is the interface you see when you load the plugin into your DAW. As you should know by now, JUCE allows you to write a custom GUI for your plugins using the JUCE GUI widgets. Commercial and other plugins have their own ways of implementing the GUI, including animated graphics, 3D, etc. How can we show this user interface using JUCE? The solution presented here is to pop up a free-floating window containing the user interface. To achieve that, you are going to explore the JUCE API a little more and define a new class.

## 18.1 AudioProcessor and AudioProcessorEditor class hierarchies

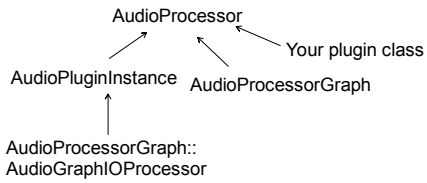


**FIGURE 18.1**
Class hierarchy for AudioProcessor and its descendants.

At this point, I will take a little detour to discuss the class hierarchy for AudioProcessors, AudioPluginInstance, AudioProcessorGraph and so on. This will allow you to better understand how to go about launching a plugin's user interface from a plugin embedded in an AudioProcessorGraph inside a node. Figure 18.1 shows the class hierarchy for the AudioProcessor classes. As you can see AudioPluginInstance and AudioProcessorGraph are both AudioProcessors. So is the application we are working on now, which

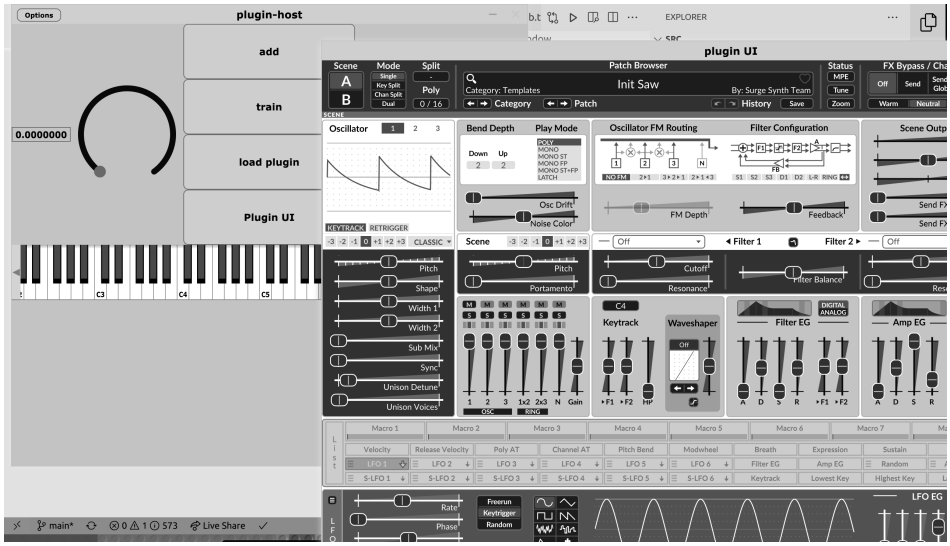


**FIGURE 18.2**
User interface for the host with the Surge XT plugin user interface showing in a separate window.

is also a plugin. This is why they all have processBlock, prepareToPlay and so on – they inherited them from AudioProcessor.

Another thing that AudioProcessor classes have are various functions for retrieving the user interface. Check out your PluginProcessor.h – there is a function override called createEditor. Look at the implementation in PluginProcessor.cpp – it creates and returns an instance of the editor class defined in your PluginEditor files. In my PluginProcessor.cpp file, the createEditor function contains one line like this:

```
return new PluginHostEditor (*this);
```

If I switch to my PluginEditor.h, I can see that it defines a PluginHostEditor class, and it takes an AudioProcessor to its constructor:

```
PluginHostEditor (PluginHostProcessor&);
```

So whenever a plugin host wants to see the user interface of my plugin, somehow it calls createEditor and createEditor calls the constructor of the user interface class and returns it.

What is the base class of the editor? In my PluginEditor.h file, the class inheritance is defined as follows:

```
class PluginHostEditor :
```

```
    public juce::AudioProcessorEditor,
    public juce::Button::Listener,
    public juce::Slider::Listener,
    private juce::MidiKeyboardState::listener
```

So in fact it has multiple base classes. The one we are interested in here is AudioProcessorEditor. If you look at the class hierarchy for AudioProcessorEditor, you will find it inherits from Component. A Component, just like any other Component in JUCE, such as a TextButton or a Slider. You could display it directly on your user interface. Similarly, if you called createEditor on any external plugin you have loaded, you will receive an AudioProcessorEditor object and it too would be a Component. So if you want to display the user interface for a plugin, you could just treat it as a Component and place it directly into your interface window.

But embedding the plugin UI in your main window is not a good idea: the lifetime of plugin UI components is a little odd, so you do not want to rely on it as a part of your main UI. The plugin UI can have a completely different aesthetic to your UI, so it does not really make sense to simply place it in your UI. And the plugin's user interface will have its own preferred size. In the following section, you will find out how to create a separate window for the plugin UI.

## 18.2 Accessing the plugin's user interface

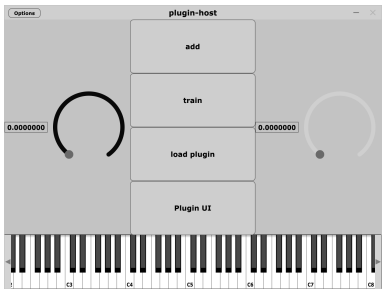


**FIGURE 18.3**
User interface for the host with a show UI button.

To create a new window for the plugin UI you will need a button on your main UI to open the plugin window and then you will need a new class to represent the window which you can instantiate and render the plugin UI into. Start by adding a new button to the main UI. The button will eventually trigger the opening of the loaded plugin's UI. As usual, you will need to call addAndMakeVisible to add it to the UI, call addListener to assign the main UI as a listener to the button then adjust the size in the resized function. My user interface ended up looking like figure 18.3.

Once you have the button displaying and you are receiving events when it is clicked, you need to figure out how to access the user interface for the plugin. Remember that you wrapped the plugin in a graph node when you implemented the AudioProcessorGraph. So you need to somehow extract the

plugin 'AudioProcessor' from the graph. Once you have access an AudioProcessor representing the plugin, you can call its createEditor function to gain access to the plugin UI.

So in PluginProcessor.h, add a new public function which will ultimately return an AudioProcessorEditor object representing the plugin's UI:

```
juce::AudioProcessorEditor* getHostedPluginEditor();
```

Then the implementation in PluginProcessor.cpp:

```
1 juce::AudioProcessorEditor* PluginHostProcessor::
      getHostedPluginEditor()
2 {
3     if (pluginNode){
4       return pluginNode->getProcessor()->createEditorIfNeeded();
5     }
6     else{
7       return nullptr;
8     }
9 }
```

On line 3 the code checks if a pluginNode is available. This will only be the case if you have already loaded a plugin via the loadPlugin button. If the pluginNode is available, it calls getProcessor on it, which will return a pointer to the AudioProcessor inside the node, or in other words, the plugin. This is odd since earlier you wrapped the plugin in a smart unique_ptr and now it is being converted back to a regular bare pointer. I think at this point you just need to be a good citizen and not call delete on that pointer. So how do you get the UI from an AudioProcessor? As mentioned earlier, you call createEditor, or better, createEditorIfNeeded which will only call createEditor if necessary. Right so you have access to the AudioProcessorEditor for your hosted plugin. Now you need to get that displayed in a window.

## 18.3 The PluginWindow class

Now for the PluginWindow class itself. Add two new empty files to your project: PluginWindow.h and PluginWindow.cpp. You will need to add at least the cpp file to your CMakeLists.txt file, as you did earlier for the NeuralNetwork class. That involves editing the target_sources command:

```
    target_sources(fm-torchknob
        PRIVATE
        src/PluginEditor.cpp
        src/PluginProcessor.cpp
        src/NeuralNetwork.cpp
```

```
        stc/PluginWindow.cpp)

```

Now put some code into the PluginWindow.h file:

```
class PluginWindow  : public juce::DocumentWindow
{
public:
    PluginWindow (juce::AudioProcessorEditor* editor);
    ~PluginWindow() override;
    void closeButtonPressed() override;
// standard macro call used to make components play nicely
JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (PluginWindow)
};
```

Here are the definitions of all those functions for PluginWindow.cpp:

```
#include "PluginWindow.h"

PluginWindow::PluginWindow (juce::AudioProcessorEditor* editor)
    :
    DocumentWindow ("plugin UI",
        juce::LookAndFeel::getDefaultLookAndFeel().
        findColour (juce::ResizableWindow::backgroundColourId),
        juce::DocumentWindow::minimiseButton |
        juce::DocumentWindow::closeButton)
{
    setSize (400, 300);

    if (editor){
        setContentOwned (editor, true);
        setResizable (editor->isResizable(), false);
    }
    setTopLeftPosition(100, 100);
    setVisible (true);
}

PluginWindow::~PluginWindow()
{
    clearContentComponent();
}

void PluginWindow::closeButtonPressed()
{
}
```

Now you should be able to use the PluginWindow class to open a window showing the plugin user interface. In PluginEditor.h, add this to the private fields:

```
std::unique_ptr<PluginWindow> pluginWindow;
```

Then in PluginEditor.cpp, implement the code to create the window when the user presses the show UI button:

```
if (btn == &showPluginUIButton){
    if (!audioProcessor.getHostedPluginEditor()) {
        DBG("showPluginUIButton no plugin");
        return;
    }
    if (pluginWindow){
        DBG("showPluginUIButton window already open");
        return;
    }
    DBG("showPluginUIButton creating new gui");
    pluginWindow =
        std::make_unique<PluginWindow>
            (audioProcessor.getHostedPluginEditor());
}
```

Note that the code deals with various possible scenarios that can occur: there is no plugin loaded yet, the plugin is loaded but the window already exists and the plugin is loaded and the window does not exist.

Try it out! You should see something like figure 18.2, which shows the Surge XT user interface rendered into a PluginWindow alongside the host user interface. Of course, you will see the user interface for your plugin of choice.

## 18.4 Closing the plugin user interface window

You may have noticed that closing the plugin user interface window is impossible. To close the window, you need to implement the closeButtonPressed function. The following instructions show you how to do this in a 'JUCE' way by defining a custom PluginWindowListener abstract class you can inherit and implement to receive the close event in the PluginEditor. First, add the following abstract class to the top of PluginWindow.h:

```
class PluginWindowListener{
public:
  PluginWindowListener(){}
  ~PluginWindowListener(){}
  virtual void pluginCloseButtonClicked() = 0;
};
```

Can you work out what makes it an abstract class instead of a normal one? That should be an easy question if you are a seasoned C++ programmer. If not, the answer is that PluginWindowListener is an abstract class because pluginCloseButtonClicked has its implementation set to '0'. You cannot instantiate an abstract class, so you cannot directly use it. This forces any class that inherits from PluginWindowListener to either be abstract itself or to provide an implementation for that function. JUCE makes extensive use of abstract classes to force

you to implement things. For example, the Button::Listener that you have been inheriting from has its buttonClicked function set to '0', forcing you to implement it in your listener.

Back to the task at hand, add a private variable to the PluginWindow class:

```
PluginWindowListener*  listener;
```

And a function to assign a listener (you can put this into PluginWindow.h):

```
void addPluginWindowListener(
      PluginWindowListener*  _listener)
    {
        listener = _listener;
    }
```

Then notify the listener when the PluginWindow's close button is pressed in PluginWindow.cpp:

```
void PluginWindow::closeButtonPressed()
{
    if (listener){
        listener->pluginCloseButtonClicked();
    }
}
```

Now you need to create a Listener that can listen to the close event and do the things necessary to remove the window. Inherit from PluginWindowListener in the PluginEditor:

```
class PluginHostEditor  :   public juce::AudioProcessorEditor,
                            public PluginWindowListener,

... in the public section:
  void pluginCloseButtonClicked() override;
...
```

Add the implementation in PluginEditor.cpp:

```
void PluginHostEditor::pluginCloseButtonClicked()
{
    // trigger delete of unique_ptr-owned object
    pluginWindow.reset();
}
```

The last step is to register the PluginHostEditor as a listener. In PluginEditor.cpp's buttonClicked function, after you create the window, register as a listener:

```
pluginWindow = std::make_unique<PluginWindow>(audioProcessor.
    getHostedPluginEditor());
pluginWindow->addPluginWindowListener(this);
```

## 18.5 Progress check

At this point, you should be able to load a plugin then click a button on your main UI that pops up a new window displaying the plugin's UI. You should also be able to close the plugin UI window, which was perhaps the hardest part!

# 19

# *From plugin host to meta-controller*

This chapter completes the implementation of the plugin meta-controller by explaining how to integrate the plugin host with the neural network code from the torchknob. The first section discusses querying plugin parameters and filtering out un-needed parameters. The following section covers creating a neural network for the plugin with the correct number of inputs and outputs. The next section discusses creating training data by setting the training knob position and selecting plugin presets. The section following that explains how to control the plugin using the neural network. The chapter concludes by mentioning possible extensions for the meta-controller, such as training dataset population, custom UI components, selective parameter control, multiple synthesizer hosting, and improved usability with state loading/saving and input features from other sources.

## 19.1 Querying plugin parameters

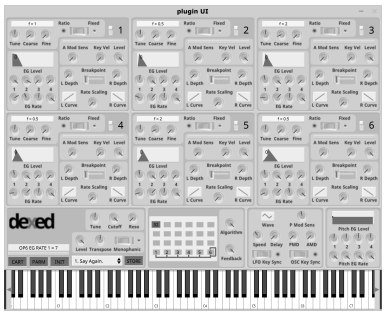


**FIGURE 19.1**
User interface for the Dexed DX-7 emulator. It has 155 parameters.

Think back to the design of the torchknob FM synthesis control system. The knob controlled two parameters at the same time. Now you will scale that control to all parameters on the plugin synthesizer. But how do you know what the parameters are on a given plugin? I have been abusing plugins with custom hosts for some time. For example, I worked with Martin Roth to develop an automatic VST plugin programmer in 2008[47]. Back then, you could ask the plugin for a list of parameters, and you would receive an array of strings and metadata for the parameters. Then you could set parameters by passing float values in the range 0-1. This is more or less how it works in JUCE now. But things are a made a little more complex be-

cause the plugin wrapper in JUCE, the AudioProcessor class[1], is plugin format agnostic. In other words, it uses the same set of parameter control functions for multiple plugin formats. Some formats have slightly more complex ways of arranging parameters, such as groups of related parameters. That makes the plugin querying code a little more complicated.

The following code prints a list of plugin parameters and their values. Add it to PluginProcessor.h and cpp as a private function:

```
void PluginHostProcessor::printPluginParameters()
{
    if (pluginNode){

        juce::Array<juce::AudioProcessorParameter *> params =
        pluginNode->getProcessor()->getParameters();

        for (const juce::AudioProcessorParameter* p : params ){
            DBG("p: " << p->getName(100) << " : " << p->getValue());
        }
    }
}
```

The '100' parameter for getName tells it to automatically crop the name to 100 characters. You have to pass this parameter. Add the code to your PluginProcessor and call printPluginParameters after loading a plugin. Here is an extract of the output I see when I load the Dexed plugin:

```
p: Cutoff : 1
p: Resonance : 0
p: Output : 1
p: MASTER TUNE ADJ : 0.5
p: ALGORITHM : 1
p: FEEDBACK : 1
p: OSC KEY SYNC : 1
p: LFO SPEED : 0.353535
p: LFO DELAY : 0
p: LFO PM DEPTH : 0
p: LFO AM DEPTH : 0
p: LFO KEY SYNC : 1
p: LFO WAVE : 0
p: TRANSPOSE : 0.5
p: P MODE SENS. : 0.428571
p: PITCH EG RATE 1 : 1
p: PITCH EG RATE 2 : 1
```

At the end of the output, I see many parameters I did not expect, for example:

```
p: MIDI CC 15|125 : 0
p: MIDI CC 15|126 : 0
p: MIDI CC 15|127 : 0
p: MIDI CC 15|128 : 0
```

[1] https://docs.juce.com/master/classAudioProcessor.html

If you develop a plugin using JUCE, JUCE adds extra parameters by default. I believe the purpose of these parameters is to expose the plugin to MIDI CC control somehow. But for now, we need to ignore them as they are not the core parameters we wish to manipulate. You can simply add a filter to the print function to remove these special parameters:

```
for (const juce::AudioProcessorParameter* p : params ){
    if (! p->getName(100).contains("MIDI CC ")){
        DBG("p: " << p->getName(100) << " : " << p->getValue());
    }
}
```

## 19.2 Create a neural network for a plugin

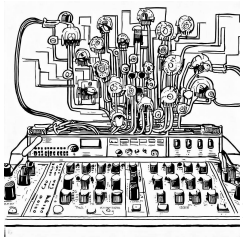

**FIGURE 19.2**
Time for a more complex neural network.

Now you have the correct set of parameters printed out, you are ready to create a neural network that can control them. The network will have one input for the torchknob and one output for each parameter. First, add a public function similar to the printPluginParameters function to PluginProcessor.h. Its purpose is to count the number of valid parameters. Here is a signature:

```
int countValidParameters();
```

You need to write the implementation, using the same filtering as printPluginParameters.

The NeuralNetwork object is stored in the PluginEditor. This makes sense as the PluginEditor receives the knob movements and then passes them to the NeuralNetwork. So after the PluginEditor calls loadPlugin, it should query the number of parameters and configure the network accordingly. So in PluginEditor.cpp::buttonClicked, after you call loadPlugin, set up the neural network:

```
fChooser.launchAsync(fileChooserFlags,
      [this](const juce::FileChooser& chooser){
        this->audioProcessor.loadPlugin(chooser.getResult());
        this->nn = NeuralNetwork(
             1,
             this->audioProcessor.countValidParameters()
        );
});
```

That creates a neural network with one input and as many outputs as the plugin has parameters. You will need a variable in the PluginEditor.h private variables

called nn and of type NeuralNetwork. Verify that your code builds and runs. You cannot train the network yet, as you must also create training data from the plugin.

## 19.3 Create training data from plugin parameters

Remember that the training data consists of inputs which are values read from the torchknob slider (or other GUI elements if you worked on the challenges from the torchknob section). The outputs consist of sets of parameter settings for the synthesizer. To generate input-output pairs for the training data, you must read the torchknob slider value and the current parameter settings. Write a public function in PluginProcessor.h/cpp that returns a vector containing the parameter values for all valid parameters on the plugin. Here is a signature for the function:

```
std::vector<float> getHostedPluginParamValues();
```

Write your own implementation, referring to the parameter querying code you can find in the printPluginParameters function above.

When the user clicks the add button, call addTrainingData on the NeuralNetwork. Here is the signature for the addTrainingData function:

```
    void addTrainingData(
        std::vector<float>inputs,
        std::vector<float>outputs);
```

You should call it in the buttonClicked function in PluginEditor.cpp. The call should look something like this:

```
nn.addTrainingData(
      {(float) superKnobTrain.getValue()},
      this->audioProcessor.getHostedPluginParamValues());
```

You should now be able to adjust the torchknob, adjust the parameters on the plugin and add training data. The effects of this will all be invisible until you activate the other dial on the right-hand side of the interface.

## 19.4 Control the plugin with the neural network

The final step is to activate the control knob on the right-hand side of the interface. The control knob should send a value into the neural network, then take the

outputs and send them to the plugin's parameters. Create a public function in PluginProcessor.h:

```
void setHostedPluginParamValues(std::vector<float> values);
```

This is the opposite of getHostedPluginParamValues. The implementation works like this:

```
void PluginHostProcessor::setHostedPluginParamValues(std::vector<
    float> values)
{
  int pind{0};
  if (pluginNode){
  juce::Array<juce::AudioProcessorParameter *> params = pluginNode->
    getProcessor()->getParameters();
    for (const juce::AudioProcessorParameter* p : params ){

      if (! p->getName(100).contains("MIDI CC ")){
        pluginNode->getProcessor()->setParameter(pind, values[pind]);
        pind++;
      }
    }
  }
}
```

That is all for now. Experiment by moving the training knob, selecting a preset on the plugin, hitting add, and repeating it a few times. Then see how well the plugin morphs between the presets. You will notice that setParameter is considered to be deprecated in JUCE. You are supposed to use AudioProcessorParameter and AudioProcessorParameterGroup, but this works for now and is simpler.

## 19.5 Extending the meta-controller

There are many possible extensions you can consider for the meta-controller:

1. Automatically populate a training dataset by iterating over the presets on the plugin, loading them and assigning them to different positions on the knob
2. Custom UI components to provide control input. Instead of one knob, you could try to implement an X,Y pad
3. Use MIDI CC data as the control input
4. Allow the user to select which parameters they want to control
5. Host and control multiple synthesizers
6. Improved usability: loading and saving state and possibly the model
7. Take input control values from another, higher dimensional source, e.g. extract features from an audio feed's spectrum or even a video feed.

## 19.6 The end of this part of the book

Excellent work! You have reached the end of this part of the book, wherein you have been working on the meta-controller. I hope you have found some unexpected places in the high-dimensional latent space of synthesizer parameters. You can, of course, use the components developed here in any way you see fit. You might find the plugin hosting a plugin concept sends you off in another direction entirely. Or you might find the many-many trainable mapping part can apply to another musical or even non-musical problem.